Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Computadores

IC-6600 Sistemas Operativos
II Semestre 2026

Tarea 3

Box de Mr. Meeseeks

Procesos con `fork()`, comunicación por `pipe()` y sincronización con semáforos en Linux

Estudiante: Nicolás Florez Jiménez
Carné: [completar]

Profesor: M.Sc. Esteban Arias-Méndez

Cartago, octubre de 2026

Índice general

1 Introducción **3**

2 Arquitectura general **3**

2.1 Organización del programa 3

2.2 Estructuras de datos 4

2.3 Comunicación por pipes 5

2.4 Semáforos 5

2.5 Ciclo de vida de un Mr. Meeseeks 6

3 Caso A: solicitudes de texto **8**

3.1 Dificultad de la solicitud 8

3.2 Duración de la simulación y desesperación 8

3.3 Modelo de resolución: Tiro Meeseeks y Ley de Dilución 8

3.4 Creación de ayudantes 9

3.5 Propagación de la solución y despedidas 11

3.6 Plan de contingencia contra el colapso 12

3.7 Código completo del caso A 13

4 Caso B: cálculos aritméticos y lógicos **16**

5 Caso C: ejecución de programas externos **17**

6 Tiempo máximo, caos planetario y medición de tiempos **19**

6.1 Caos planetario 19

6.2 Qué se mide 21

7 Uso de Git **22**

8 Compilación y uso **22**

9 Corridas de ejemplo **22**

9.1 Caso A con dificultad indicada por el usuario 23

9.2 Caso A con la dificultad a la suerte del Meeseeks 25

9.3 Caso A imposible y plan de contingencia 27

9.4 Caso B: cálculos 29

9.5 Caso C: programas externos 30

9.6 Caos planetario 31

10 0x02: experimentos y análisis **33**

10.1 Metodología 33

10.2 E1: costo en función de la dificultad 33

10.3 E2: duración aleatoria frente a duración fija 35

10.4 E3: comparación de los tres casos 37

10.5 E4: plan de contingencia 38

10.6 E5: sensibilidad a la ganancia de dilución 39

10.7 E6: tareas que superan el tiempo máximo 40

11 Conclusiones **42**

1 Introducción

En el episodio *Meeseeks and Destroy* de la serie *Rick and Morty*, una caja con un botón invoca a un Mr. Meeseeks: un ser que existe únicamente para cumplir una solicitud y que desaparece en cuanto lo logra. Cuando la tarea es difícil, el Meeseeks invoca a otros Meeseeks para que lo ayuden, y estos a otros más, hasta que la situación se sale de control. La analogía con un sistema operativo es directa: cada Meeseeks es un proceso, la caja es el proceso que los crea, y la proliferación sin control es una *fork bomb*.

Esta tarea implementa esa caja en C sobre Linux como un único programa. El proceso original (la *Box*) recibe solicitudes del usuario y, para cada una, crea con `fork()` un Mr. Meeseeks raíz. Se atienden tres tipos de solicitud: (A) tareas textuales simuladas cuya dificultad puede requerir un árbol de ayudantes, (B) cálculos aritméticos o lógicos y (C) la ejecución de programas externos. Los Meeseeks se comunican exclusivamente por *pipes*; los contadores compartidos de niveles, instancias y estadísticas viven en memoria compartida y se protegen con semáforos POSIX administrados por el kernel. Un tiempo máximo configurable decreta el *caos planetario* y termina todos los procesos de la solicitud.

El documento describe la arquitectura del programa (sección 2), cada caso de solicitud con el código que lo implementa (secciones 3 a 5), el manejo del tiempo máximo y la forma de medir los tiempos (sección 6), el uso de Git (sección 7) y corridas de ejemplo (sección 9). La sección 10 corresponde a la parte 0x02: seis experimentos que comparan el comportamiento de los casos y escenarios, con su análisis. Al final se presentan las conclusiones.

2 Arquitectura general

2.1 Organización del programa

Todo el código está en un solo archivo, `src/meeseeks.c`, y todos los procesos ejecutan ese mismo código. El proceso que se ejecuta desde la terminal toma el papel de la Box (función `main`); cada proceso creado con `fork()` entra a la función `meeseeks_run`, que es el punto de entrada de cualquier Mr. Meeseeks, sea la raíz ($N = 1$, $i = 1$) o un ayudante. La figura 1 muestra la relación entre los procesos, los pipes y la memoria compartida.

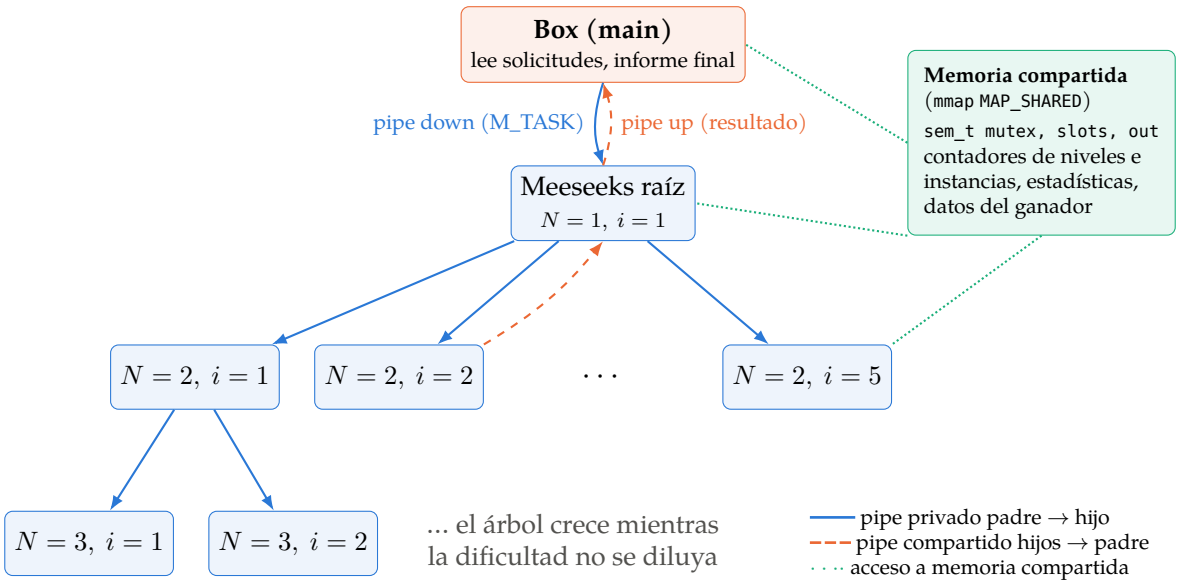


Figura 1: Procesos, pipes y memoria compartida. Cada flecha sólida es un pipe distinto; los hijos de un mismo padre comparten un único pipe de subida.

2.2 Estructuras de datos

Los mensajes entre procesos tienen tamaño fijo. La estructura `msg_t` ocupa 320 bytes, menos que `PIPE_BUF` (4096 bytes en Linux), por lo que POSIX garantiza que cada `write()` sobre el pipe es atómico: aunque cinco hermanos escriban al mismo tiempo en el pipe de subida, los mensajes nunca se entremezclan. La identidad de un Meeseeks es la tupla $(pid, ppid, N, i)$ que pide el enunciado.

Código 1: Identidad de un Meeseeks y formato de los mensajes (`src/meeseeks.c`, línea 94)

```
94 /* Identidad de un Meeseeks: (pid, ppid, N, i) */
95 typedef struct {
96     pid_t  pid, ppid;
97     int    N, i;
98     double d;            /* dificultad con la que trabajo */
99 } who_t;
100
101 /* Mensaje de tamano fijo (< PIPE_BUF = 4096 bytes => escritura atomica) */
102 typedef struct {
103     int    type;         /* mtype_t                                      */
104     int    kind;         /* kind_t                                       */
105     double difficulty;   /* dificultad (ya diluida); < 0 = "a la suerte" */
106     double duration;     /* duracion base definida por el usuario (<=0 rand)*/
107     double path_sleep;   /* espera simulada acumulada desde la raiz      */
108     int    ok;           /* M_RESULT: 1 exito, 0 fallo                   */
109     who_t  solver;       /* quien resolvio / quien envia                 */
110     char   text[MAX_TEXT];/* solicitud (M_TASK) o resultado (M_RESULT)   */
111 } msg_t;
```

La memoria compartida contiene solo contadores y estadísticas; ningún mensaje viaja por ella, como exige el enunciado. Se crea una vez con `mmap(MAP_SHARED|MAP_ANONYMOUS)` antes de cualquier `fork()`, de modo que todos los procesos descendientes ven la misma página física.

Código 2: Región compartida entre todos los procesos (`src/meeseeks.c`, línea 113)

```
113 /* Region compartida entre TODOS los procesos (contadores, no mensajes) */
114 typedef struct {
115     sem_t  mutex;                   /* protege todo lo de abajo           */
116     sem_t  slots;                   /* Meeseeks vivos permitidos          */
117     sem_t  out;                     /* serializa la salida en pantalla    */
118     long   created;                 /* Meeseeks creados en esta solicitud */
119     long   alive;                   /* Meeseeks vivos ahora               */
120     long   max_alive;               /* pico de Meeseeks vivos             */
121     long   failed_forks;            /* fork() que fallaron                */
122     long   capped;                  /* veces que se toco el tope total    */
123     int    max_level;               /* nivel N mas profundo alcanzado     */
124     long   level_count[MAX_LEVELS + 1];/* instancias creadas por nivel      */
125     double sum_sleep;               /* suma de esperas simuladas          */
126     double sum_active;              /* suma de tiempo real sin esperas    */
127     double max_path_sleep;          /* espera simulada del camino mas largo*/
128     double root_d;                  /* dificultad efectiva de la raiz     */
129     int    solved;                  /* 1 si alguien resolvio              */
130     who_t  solver;                  /* datos del que resolvio             */
131     double solver_path_sleep;       /* espera simulada en su camino       */
132 } shared_t;
```

2.3 Comunicación por pipes

Cada vez que un Meeseeks crea a un ayudante abre un pipe nuevo (*down*) que solo ellos dos conocen: por él el padre le envía en privado la solicitud (`M_TASK`) y, si hace falta, la orden de desaparecer (`M_STOP`). En sentido contrario, todos los hijos de un mismo padre escriben en un único pipe de subida (*up*) que el padre lee con `poll()`. Inmediatamente después del `fork()` el hijo cierra todos los descriptores que no le pertenecen (los del abuelo, el extremo de lectura del pipe de subida y los pipes privados de sus hermanos), lo que evita fugas de descriptores al crecer el árbol. Los mensajes posibles son:

Mensaje	Dirección	Significado
`M_TASK`	padre → hijo	solicitud a atender, con la dificultad ya diluida
`M_STOP`	padre → hijo	la tarea ya se cumplió: despedirse y desaparecer
`M_SOLVED`	hijo → padre	la tarea fue resuelta; lleva la tupla del ganador
`M_FAILED`	hijo → padre	la rama no pudo (se activó la contingencia)
`M_RESULT`	Meeseeks → Box	resultado de un cálculo o de un programa externo

2.4 Semáforos

Se usan tres semáforos POSIX sin nombre (`sem_t`) inicializados con `pshared = 1`, lo que los hace válidos entre procesos porque residen en la memoria compartida. El kernel administra la espera mediante *futex*: un proceso bloqueado en `sem_wait` no consume CPU.

- `mutex` (valor inicial 1): exclusión mutua sobre todos los contadores. Cada Meeseeks lo toma al registrarse (`register_self`), al desaparecer (`vanish`), al reservar la creación de un ayudante y al reclamar la victoria.

- `slots` (valor inicial `-a`, por defecto 128): semáforo contador que limita cuántos Meeseeks pueden estar *trabajando* a la vez. Es la primera capa del plan de contingencia (sección 3.6).
- `out` (valor inicial 1): serializa la impresión. Cada línea se arma completa en memoria y se escribe con un solo `write()`, por lo que nunca se mezclan dos mensajes en pantalla. Se usa `sem_timedwait` con un límite de 200 ms para que un proceso terminado abruptamente mientras imprimía no bloquee a los demás.

Código 3: Registro de un Meeseeks en los contadores compartidos (`src/meeseeks.c`, línea 273)

```
273 /* Se registra en los contadores compartidos (seccion critica) */
274 static void register_self(void)
275 {
276     sem_wait(&SH->mutex);
277     SH->alive++;
278     if (SH->alive > SH->max_alive) SH->max_alive = SH->alive;
279     if (ME.N <= MAX_LEVELS) SH->level_count[ME.N]++;
280     if (ME.N > SH->max_level) SH->max_level = ME.N;
281     sem_post(&SH->mutex);
282 }
```

Código 4: Desaparición: actualiza estadísticas y libera su *slot* (`src/meeseeks.c`, línea 284)

```
284 /* Se despide del mundo: actualiza estadisticas, libera su "slot" y muere.
285  * Se usa _exit para no vaciar dos veces buffers de stdio heredados.        */
286 static void vanish(int code)
287 {
288     double life = now_s() - BORN_AT;
289     sem_wait(&SH->mutex);
290     SH->alive--;
291     SH->sum_sleep  += SLEPT;
292     SH->sum_active += (life - SLEPT > 0 ? life - SLEPT : 0);
293     sem_post(&SH->mutex);
294     if (HOLDS_SLOT) sem_post(&SH->slots);
295     _exit(code);
296 }
```

2.5 Ciclo de vida de un Mr. Meeseeks

La figura 2 resume los estados por los que pasa un Meeseeks que atiende una solicitud de texto. Lo primero que hace al nacer es saludar con el mensaje exigido, `Hi, I'm Mr. Meeseeks! Look at Meeeee. (pid, ppid, N, i)`; después lee la solicitud de su pipe privado. Todos los mensajes que imprime un Meeseeks llevan el prefijo `[pid | ppid | N i]`.

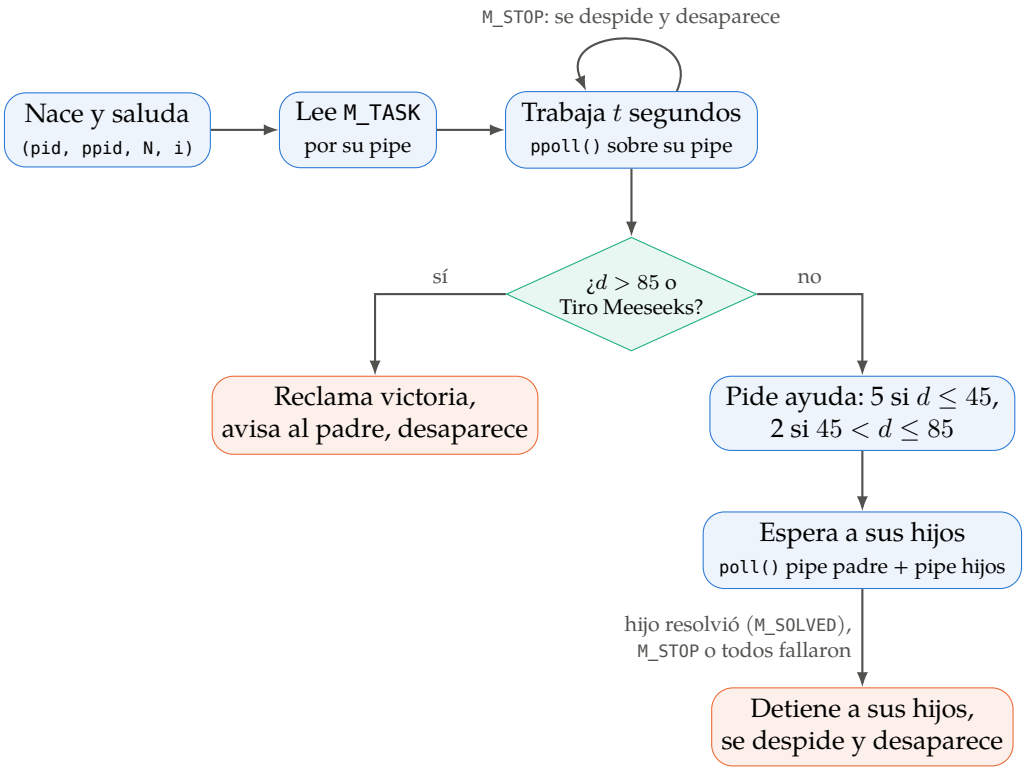


Figura 2: Estados de un Mr. Meeseeks que atiende una solicitud de texto.

Mientras simula su trabajo, el Meeseeks no duerme con `sleep()`: espera con `ppoll()` sobre su pipe privado, de manera que si otro Meeseeks resuelve la tarea y le llega `M_STOP`, se despide y desaparece de inmediato en lugar de terminar su espera.

Código 5: Trabajo simulado que sigue escuchando el pipe (`src/meeseeks.c`, línea 298)

```
298 /* Duerme "dur" segundos simulando trabajo, pero sigue escuchando su pipe:
299  * si llega M_STOP (o el padre desaparece) devuelve 1 de inmediato.
300  * Se usa ppoll() porque poll() solo acepta milisegundos enteros y
301  * redondeaba hacia arriba duraciones muy cortas.                          */
302 static int work_or_stop(int in_fd, double dur)
303 {
304     double start = now_s(), end = start + dur;
305     for (;;) {
306         double rem = end - now_s();
307         if (rem <= 0) break;
308         struct pollfd p = { in_fd, POLLIN, 0 };
309         struct timespec to;                /* ppoll: resolucion de ns */
310         to.tv_sec  = (time_t)rem;
311         to.tv_nsec = (long)((rem - (double)to.tv_sec) * 1e9);
312         int r = ppoll(&p, 1, &to, NULL);
313         if (r < 0 && errno == EINTR) continue;
314         if (r > 0) {
315             msg_t m;
316             int k = read_msg(in_fd, &m);
317             if (k <= 0 || m.type == M_STOP) { SLEPT += now_s() - start; return 1; }
318         }
319     }
```

```
320     SLEPT += now_s() - start;
321     return 0;
322 }
```

3 Caso A: solicitudes de texto

3.1 Dificultad de la solicitud

La dificultad es un valor $d \in [0, 100]$, donde 100 es muy fácil y 0 imposible. Cuando el usuario escribe una solicitud de texto, la Box le pregunta la dificultad (caso 1). Si el usuario presiona Enter, la Box envía $d = -1$ y el propio Meeseeks raíz la estima a su suerte (caso 2) con una distribución Beta$(4, 2)$ escalada a $[0, 100]$. Se eligió esta distribución porque su media es $2/3$ ($d \approx 66.7$): la mayoría de las solicitudes cotidianas son de dificultad media o fácil, pero la cola izquierda deja una probabilidad pequeña de tareas muy difíciles, como en la serie. Se genera sin bibliotecas externas con el estadístico de orden: el cuarto menor de cinco números uniformes sigue una Beta$(4, 2)$.

Código 6: Dificultad a la suerte del Meeseeks: Beta(4,2) (`src/meeseeks.c`, línea 188)

```
188 /* Beta(4,2) por estadisticos de orden: el 4.o menor de 5 uniformes.
189  * Media 2/3: la mayoria de las solicitudes cotidianas son mas bien faciles,
190  * pero la cola izquierda deja espacio para tareas muy dificiles.          */
191 static double beta_4_2(void)
192 {
193     double u[5];
194     for (int k = 0; k < 5; k++) u[k] = urand();
195     for (int a = 1; a < 5; a++)             /* insercion: 5 elementos */
196         for (int b = a; b > 0 && u[b - 1] > u[b]; b--) { double t = u[b]; u[b] = u[b - 1]; u[b -
        1] = t; }
197     return u[3];
198 }
```

3.2 Duración de la simulación y desesperación

Cada Meeseeks trabaja un tiempo uniforme entre 0.5 y 5 s, o el valor base que indique el usuario. Ese tiempo se multiplica por un factor de *desesperación* que crece con el nivel: $1 + 0.1\,(N - 1)$ para tareas posibles y $1 + 0.5\,(N - 1)$ para tareas imposibles ($d = 0$). Así, como pide el enunciado, ante una tarea imposible cada nueva generación de Meeseeks tarda notablemente más que la anterior.

3.3 Modelo de resolución: Tiro Meeseeks y Ley de Dilución

Después de trabajar, el Meeseeks decide si cumplió la solicitud con las reglas del enunciado más dos ideas propias:

1. **Tareas fáciles** ($85 < d \leq 100$): las resuelve un solo Meeseeks.
2. **Tiro Meeseeks** ($0 < d \leq 85$): antes de pedir ayuda, el Meeseeks intenta la tarea por su cuenta y lo logra con probabilidad $p(d) = (d/100)^3$. La forma cúbica castiga mucho la dificultad: con $d = 80$ tiene 51 % de éxito, con $d = 50$ solo 12.5 %, con $d = 20$ menos de 1 %.

3. **Pedir ayuda**: si falla el tiro, invoca 5 ayudantes cuando $d \leq 45$ (nivel $N+1$, instancias $i = 1, \ldots, 5$) o 2 ayudantes cuando $45 < d \leq 85$ ($i = 1, 2$).

4. **Ley de Dilución Meeseeks**: cada ayudante recibe una versión más sencilla de la tarea, como si se repartiera entre ellos. La nueva dificultad sigue un crecimiento logístico con ruido multiplicativo:

$$d' = \min\Big(100,\ \big[d + r\,d\,(1 - \tfrac{d}{100})\big]\, e^{0.08Z}\Big), \qquad r = g\sqrt{k}, \quad Z \sim \mathcal{N}(0,1),$$

   donde k es la cantidad de ayudantes y g la ganancia (por defecto 0.5). Con más ayudantes la tarea se simplifica más rápido ($r \approx 1.12$ con 5 ayudantes y $r \approx 0.71$ con 2). Es una forma de *añejamiento* (*aging*): igual que en planificación la prioridad de un proceso crece mientras espera, aquí la facilidad de la tarea crece con cada generación hasta superar 85.

La propiedad clave de la ley es que $d = 0$ es un punto fijo: como el término de crecimiento y el ruido son proporcionales a d, una tarea imposible sigue siendo imposible sin importar cuántos Meeseeks se invoquen, que es exactamente lo que pide el enunciado. En cambio, cualquier $d > 0$ por pequeño que sea termina diluyéndose: cerca de 0 la dificultad se multiplica por $1 + r \approx 2.1$ en cada generación, de modo que el número de niveles necesarios crece como $\log(1/d)$ y el número de Meeseeks crece en forma exponencial con esos niveles. La sección 10 mide este comportamiento.

Código 7: Ley de Dilución Meeseeks (`src/meeseeks.c`, línea 328)

```
328 /* "Ley de la Dilucion Meeseeks" (inspirada en el envejecimiento/aging):
329  *   d' = [ d + r*d*(1 - d/100) ] * exp(0.08*Z),   r = g*sqrt(k)
330  * Es un crecimiento logistico: mientras mas ayudantes (k) se reparten la
331  * tarea, mas rapido sube la "facilidad" de cada parte. El ruido es
332  * multiplicativo, por lo que d = 0 es un punto fijo: lo imposible jamas se
333  * vuelve posible, sin importar cuantos Meeseeks se invoquen.              */
334 static double dilute(double d, int k)
335 {
336     if (d <= 0.0) return 0.0;
337     double r  = CFG.gain * sqrt((double)k);
338     double dn = (d + r * d * (1.0 - d / 100.0)) * exp(0.08 * gauss());
339     if (dn > 100.0) dn = 100.0;
340     return dn;
341 }
```

Código 8: Probabilidad del Tiro Meeseeks (`src/meeseeks.c`, línea 343)

```
343 /* "Tiro Meeseeks": probabilidad de que un Meeseeks, solo, logre una tarea
344  * de dificultad d <= 85 sin pedir ayuda. Cubica: castiga mucho lo dificil. */
345 static double self_success_p(double d)
346 {
347     double x = d / 100.0;
348     return x * x * x;
349 }
```

3.4 Creación de ayudantes

La función `spawn_helper` pide un *slot* al semáforo contador, reserva la creación en el contador compartido (y verifica el tope total), crea el pipe privado, hace `fork()` y le envía al hijo la solicitud con la dificultad diluida. El hijo cierra los descriptores ajenos y entra a `meeseeks_run` con $N+1$ y su número de instancia.

Código 9: Creación de un ayudante (`src/meeseeks.c`, línea 355)

```
355 /* Intenta invocar un ayudante. Devuelve 1 si lo creo, 0 si no hay slot
356  * libre en este momento (reintentar), -1 si se alcanzo el tope o fork fallo.*/
357 static int spawn_helper(const msg_t *task, double child_d, double path,
358                          int in_fd, int out_fd, int up[2],
359                          kid_t *kids, int nk)
360 {
361     /* Contingencia 1: semaforo contador de Meeseeks vivos */
362     if (sem_trywait(&SH->slots) != 0) return 0;
363 
364     /* Contingencia 2: tope total de Meeseeks por solicitud */
365     sem_wait(&SH->mutex);
366     if (CFG.max_total > 0 && SH->created >= CFG.max_total) {
367         SH->capped++;
368         sem_post(&SH->mutex);
369         sem_post(&SH->slots);
370         return -1;
371     }
372     SH->created++;
373     sem_post(&SH->mutex);
374 
375     int down[2];
376     if (pipe(down) < 0) goto fail;
377 
378     fflush(NULL);
379     pid_t pid = fork();
380     if (pid < 0) { close(down[0]); close(down[1]); goto fail; }
381 
382     if (pid == 0) {
383         /* Hijo: cierra los descriptores que no le pertenecen */
384         close(in_fd);
385         close(out_fd);
386         close(up[0]);
387         close(down[1]);
388         for (int j = 0; j < nk; j++) if (kids[j].alive) close(kids[j].wfd);
389         meeseeks_run(down[0], up[1], ME.N + 1, nk + 1);   /* N+1, i = 1..k */
390         _exit(0);
391     }
392 
393     /* Padre: le pasa la solicitud por su pipe privado */
394     close(down[0]);
395     msg_t m = *task;
396     m.type       = M_TASK;
397     m.difficulty = child_d;
398     m.path_sleep = path;
399     write_msg(down[1], &m);
400     kids[nk].pid = pid; kids[nk].wfd = down[1]; kids[nk].alive = 1;
401     return 1;
402 
403 fail:
404     sem_wait(&SH->mutex);
405     SH->created--;
406     SH->failed_forks++;
407     sem_post(&SH->mutex);
408     sem_post(&SH->slots);
```

```
409     return -1;
410 }
```

3.5 Propagación de la solución y despedidas

Cuando varios Meeseeks trabajan en paralelo es posible que dos o más logren la tarea casi al mismo tiempo. Para que haya un único ganador, el que tiene éxito debe reclamar la victoria dentro de una sección crítica: el primero que toma el `mutex` y encuentra `solved = 0` gana; los demás reportan que alguien se les adelantó y esperan la orden de desaparecer.

Código 10: Victoria exclusiva (`src/meeseeks.c`, línea 412)

```
412 /* Reclama la victoria en la memoria compartida (seccion critica).
413  * Solo el primero gana; devuelve 1 si este Meeseeks es el ganador.
414  * Si otro ya gano, *other recibe sus datos.                          */
415 static int claim_victory(who_t w, double path, who_t *other)
416 {
417     int won = 0;
418     sem_wait(&SH->mutex);
419     if (!SH->solved) { SH->solved = 1; SH->solver = w; SH->solver_path_sleep = path; won = 1; }
420     else if (other) *other = SH->solver;
421     sem_post(&SH->mutex);
422     return won;
423 }
```

El ganador avisa a su padre por el pipe de subida (`M_SOLVED` con su tupla). El padre envía `M_STOP` por el pipe privado de cada uno de sus hijos (los hermanos del ganador), espera con `waitpid` a que se despidan, se despide él mismo y reenvía el mensaje a su propio padre. La orden de desaparecer se propaga recursivamente hacia abajo por las demás ramas, y la noticia del éxito sube hasta la raíz, que finalmente informa al usuario qué Meeseeks resolvió la solicitud. Luego el control vuelve a la Box.

Código 11: Detener a todos los hijos y esperarlos (`src/meeseeks.c`, línea 440)

```
440 /* Envia M_STOP por el pipe privado de cada hijo vivo y espera a que mueran */
441 static void stop_all_kids(kid_t *kids, int nk)
442 {
443     msg_t s;
444     memset(&s, 0, sizeof s);
445     s.type = M_STOP;
446     for (int j = 0; j < nk; j++)
447         if (kids[j].alive) write_msg(kids[j].wfd, &s);   /* EPIPE se ignora */
448     for (int j = 0; j < nk; j++) {
449         if (kids[j].alive) {
450             while (waitpid(kids[j].pid, NULL, 0) < 0 && errno == EINTR) ;
451             kids[j].alive = 0;
452         }
453         close(kids[j].wfd);
454     }
455 }
```

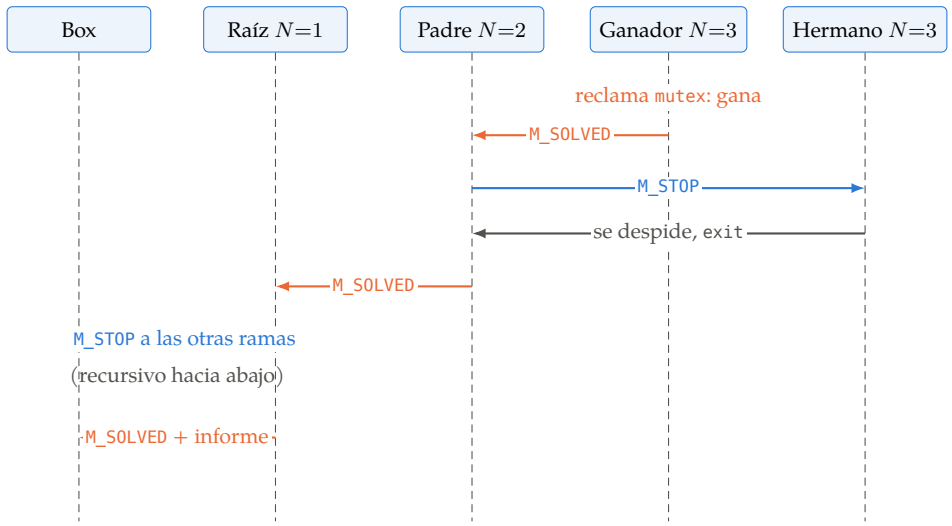


Figura 3: Secuencia de mensajes cuando un Meeseeks de nivel 3 resuelve la tarea.

3.6 Plan de contingencia contra el colapso

Una solicitud imposible ($d = 0$) siempre pide 5 ayudantes por Meeseeks: sin control serían 5^{N-1} procesos en el nivel N, una *fork bomb*. El programa aplica cuatro capas, documentadas también al ejecutarlo:

1. **Semáforo de *slots*** (`-a`, 128 por defecto): un Meeseeks solo puede trabajar si tomó un *slot*; para crear un ayudante hay que obtener uno con `sem_trywait`, y si no hay, el padre espera con `poll()` (sin dejar de atender su pipe) y reintenta. Esto limita la carga de CPU y el ritmo de creación.
2. **Tope total por solicitud** (`-m`, 3000 por defecto): el contador compartido `created` se incrementa *antes* del `fork()` dentro de la sección crítica. Si alcanzó el tope, no se crean más Meeseeks; las hojas que ya no pueden pedir ayuda reportan `M_FAILED` y la falla se propaga hasta la raíz, que informa que la solicitud no pudo completarse. El número total de procesos vivos queda acotado por este tope, muy por debajo de `pid_max` (32 768 en el equipo de pruebas).
3. **Fallas de `fork()`**: si el sistema niega un proceso (`EAGAIN`), se deshace la reserva y se trata igual que el tope.
4. **Caos planetario** (`-T`, 300 s por defecto): la última línea de defensa, descrita en la sección 6.

Un interbloqueo encontrado durante el desarrollo. En la primera versión, cada Meeseeks conservaba su *slot* durante toda su vida, incluso mientras esperaba a sus ayudantes. Con una tarea imposible y un tope de 400, el programa se quedó congelado: los 128 procesos vivos eran padres esperando un *slot* para crear a su siguiente ayudante, y ninguno lo liberaba porque todos esperaban. Es un caso de libro de las cuatro condiciones de Coffman (exclusión mutua, retención y espera, no expropiación y espera circular). La corrección fue romper la retención y espera: el *slot* representa a un Meeseeks *trabajando*; en el momento en que un Meeseeks decide pedir ayuda y pasa a esperar, libera su *slot* (`HOLDS_SLOT = 0`). Desde entonces siempre hay *slots* para las hojas y la solicitud termina por el tope o por el tiempo máximo.

3.7 Código completo del caso A

Código 12: Atención de una solicitud de texto (`src/meeseeks.c`, línea 457)

```
457 static void handle_text(int in_fd, int out_fd, msg_t *t)
458 {
459     /* Caso 2: la dificultad se deja a la suerte del Meeseeks creado */
460     double d = t->difficulty;
461     if (d < 0.0) {
462         d = 100.0 * beta_4_2();
463         say("Nadie me dijo que tan dificil es... lo dejo a mi suerte (Beta(4,2)): dificultad %.2f
      ", d);
464     }
465     ME.d = d;
466     if (ME.N == 1) { sem_wait(&SH->mutex); SH->root_d = d; sem_post(&SH->mutex); }
467 
468     /* Duracion de la simulacion: 0.5..5 s o la del usuario; con cada nivel
469      * los Meeseeks se desesperan y tardan mas (mucho mas si d = 0).        */
470     double base = t->duration > 0.0 ? t->duration : 0.5 + 4.5 * urand();
471     double desp = (d <= 0.0) ? 1.0 + 0.5 * (ME.N - 1) : 1.0 + 0.1 * (ME.N - 1);
472     double dur  = base * desp;
473 
474     say("Recibi por mi pipe la solicitud \"%s\" (dificultad %.2f). Ooooh, puedo hacerlo!
      Trabajando %.2f s...",
475         t->text, d, dur);
476 
477     if (work_or_stop(in_fd, dur)) {
478         say("Me avisan por el pipe que la solicitud ya fue cumplida. Adios! *puff*");
479         vanish(0);
480     }
481     double path = t->path_sleep + dur;
482     record_path(path);
483 
484     /* ----- Decision: lo resuelvo yo o pido ayuda ----- */
485     int mine = 0;
486     if (d > EASY_LIMIT) {
487         mine = 1;
488     } else if (d > 0.0) {
489         double p = self_success_p(d), roll = urand();
490         if (roll < p) {
491             mine = 1;
492             say("Tiro Meeseeks exitoso! (probabilidad %.4f, saque %.4f). Lo logre sin pedir ayuda
      .", p, roll);
493         }
494     }
495 
496     who_t me = ME;
497     me.pid = getpid();
498     me.ppid = getppid();
499 
500     if (mine) {
501         who_t other;
502         if (!claim_victory(me, path, &other)) {
503             say("Lo logre, pero Mr. Meeseeks (pid %d, N=%d, i=%d) se me adelanto. Espero la orden
       de desaparecer.",
```

```
                other.pid, other.N, other.i);
            wait_for_stop(in_fd);
            say("Me avisan por el pipe que la solicitud ya fue cumplida. Adios! *puff*");
            vanish(0);
        }
        say("Solicitud completada: \"%s\"! Gracias por dejarme servirle. Adios! *puff*", t->text);
        if (ME.N == 1)
            say("INFORME FINAL: la solicitud \"%s\" fue resuelta por Mr. Meeseeks (pid %d, ppid %d, N=%d, i=%d).",
                t->text, me.pid, me.ppid, me.N, me.i);
        msg_t r;
        memset(&r, 0, sizeof r);
        r.type = M_SOLVED; r.kind = K_TEXT; r.solver = me; r.path_sleep = path; r.ok = 1;
        write_msg(out_fd, &r);
        vanish(0);
    }

    /* ----- Pedir ayuda: 5 ayudantes si d <= 45, 2 si 45 < d <= 85 ----- */
    int want = (d <= HARD_LIMIT) ? 5 : 2;
    if (d <= 0.0)
        say("Esta tarea es IMPOSIBLE (dificultad %.2f)! La existencia es dolor! Invoco %d Mr. Meeseeks mas (nivel %d)...",
            d, want, ME.N + 1);
    else
        say("Esto es mas dificil de lo que pense (dificultad %.2f). Necesito ayuda! Invoco %d Mr. Meeseeks (nivel %d)...",
            d, want, ME.N + 1);

    /* Deja de "trabajar" y pasa a esperar a sus ayudantes: libera su slot.
     * Sin esto, con tareas muy dificiles todos los procesos vivos terminan
     * siendo padres que esperan un slot que nadie libera (interbloqueo).   */
    sem_post(&SH->slots);
    HOLDS_SLOT = 0;

    int up[2];
    if (pipe(up) < 0) { say("No pude crear el pipe de mis ayudantes."); goto failed_noup; }

    kid_t kids[MAX_KIDS];
    int nk = 0, finished = 0, capped_said = 0;
    int result = 0;              /* 0 pendiente, 1 resuelto, 2 STOP, -1 fallo */
    msg_t win;
    memset(&win, 0, sizeof win);

    while (result == 0) {
        /* Crear ayudantes pendientes, si hay slot libre */
        if (nk < want) {
            double cd = dilute(d, want);
            int s = spawn_helper(t, cd, path, in_fd, out_fd, up, kids, nk);
            if (s == 1) { nk++; continue; }
            if (s < 0) {
                if (!capped_said) {
                    say("Contingencia: no puedo invocar mas Mr. Meeseeks (tope de %ld por solicitud o fork fallido).",
                        CFG.max_total);
```

```
                    capped_said = 1;
                }
                want = nk;
                if (nk == 0 || finished == nk) { result = -1; break; }
            }
        }

        struct pollfd p[2] = { { in_fd, POLLIN, 0 }, { up[0], POLLIN, 0 } };
        int r = poll(p, 2, nk < want ? 25 : -1);
        if (r < 0) { if (errno == EINTR) continue; break; }

        if (p[0].revents) {                     /* mensaje de mi padre */
            msg_t m;
            int k = read_msg(in_fd, &m);
            if (k <= 0 || m.type == M_STOP) { result = 2; break; }
        }
        if (p[1].revents & POLLIN) {            /* mensaje de un hijo  */
            msg_t m;
            if (read_msg(up[0], &m) == 1) {
                if (m.type == M_SOLVED) { win = m; result = 1; break; }
                if (m.type == M_FAILED) {
                    finished++;
                    for (int j = 0; j < nk; j++)
                        if (kids[j].alive && kids[j].pid == m.solver.pid) {
                            waitpid(kids[j].pid, NULL, 0);
                            kids[j].alive = 0;
                        }
                    if (nk == want && finished == nk) { result = -1; break; }
                }
            }
        }
    }

    if (result == 1) {
        say("Mi ayudante reporta que Mr. Meeseeks (pid %d, ppid %d, N=%d, i=%d) resolvio la
    solicitud! Aviso a mis hijos para que desaparezcan.",
            win.solver.pid, win.solver.ppid, win.solver.N, win.solver.i);
        stop_all_kids(kids, nk);
        say("Mi trabajo aqui termino. Gracias por permitirme servirle. Adios! *puff*");
        if (ME.N == 1)
            say("INFORME FINAL: la solicitud \"%s\" fue resuelta por Mr. Meeseeks (pid %d, ppid %
    d, N=%d, i=%d).",
                t->text, win.solver.pid, win.solver.ppid, win.solver.N, win.solver.i);
        win.type = M_SOLVED;
        write_msg(out_fd, &win);
        close(up[0]); close(up[1]);
        vanish(0);
    }
    if (result == 2) {
        stop_all_kids(kids, nk);
        say("Me avisan por el pipe que la solicitud ya fue cumplida. Adios! *puff*");
        close(up[0]); close(up[1]);
        vanish(0);
    }
    stop_all_kids(kids, nk);
```

```
607     close(up[0]); close(up[1]);
608
609 failed_noup:
610     say("Ninguno de mis ayudantes pudo con \"%s\". La existencia es dolor! Me rindo. *puff*", t->
        text);
611     if (ME.N == 1)
612         say("INFORME FINAL: la solicitud \"%s\" NO pudo completarse.", t->text);
613     {
614         msg_t f;
615         memset(&f, 0, sizeof f);
616         f.type = M_FAILED; f.kind = K_TEXT; f.solver = me; f.path_sleep = path;
617         write_msg(out_fd, &f);
618     }
619     vanish(1);
620 }
```

4 Caso B: cálculos aritméticos y lógicos

Una solicitud que empieza con `calc` (o con `=`) la atiende un solo Meeseeks, que evalúa la expresión con un analizador recursivo descendente escrito para esta tarea. No hay simulación de espera: el tiempo medido es el de crear el proceso, pasarle la expresión por el pipe, evaluarla y devolver el resultado. La gramática, de menor a mayor precedencia, es:

Nivel	Producción
`or`	`and ( "||" and )*`
`and`	`eq ( "&&" eq )*`
`eq`	`rel ( ("==" | "!=") rel )*`
`rel`	`add ( ("<" | "<=" | ">" | ">=") add )*`
`add`	`mul ( ("+" | "-") mul )*`
`mul`	`unary ( ("*" | "/" | "%") unary )*`
`unary`	`("-" | "+" | "!") unary | power`
`power`	`primary ( "^" unary )?`
`primary`	`número | "(" or ")" | true | false | sqrt(...) | abs(...)`

Consideraciones especiales:

- Los números son de punto flotante (`double`); un resultado entero se imprime sin decimales.
- La potencia es asociativa a la derecha ($2\text{^}3\text{^}2 = 512$) y el menos unario tiene menor precedencia que ella ($-2\text{^}2 = -4$), como en matemática.
- Los operadores relacionales y lógicos devuelven 1 o 0, y cualquier valor distinto de 0 es verdadero, igual que en C.
- Se reportan como error, con la posición del problema: símbolos inesperados, paréntesis sin cerrar, texto sobrante, división o módulo entre cero, raíz de negativos y resultados no finitos. En ese caso la solicitud se registra como *no completada*.

Código 13: Nivel multiplicativo del analizador (con detección de división entre cero) (`src/meeseeks.c`, línea 716)

```
716 static double p_mul(parser_t *p)
717 {
718     double v = p_unary(p);
719     for (;;) {
720         if (p->err) return 0;
721         if (p_eat(p, "*")) { v *= p_unary(p); }
722         else if (p_eat(p, "/")) {
723             double r = p_unary(p);
724             if (r == 0) { p_fail(p, "division entre cero"); return 0; }
725             v /= r;
726         } else if (p_eat(p, "%")) {
727             double r = p_unary(p);
728             if (r == 0) { p_fail(p, "modulo entre cero"); return 0; }
729             v = fmod(v, r);
730         } else return v;
731     }
732 }
```

Código 14: Punto de entrada del evaluador (`src/meeseeks.c`, línea 783)

```
783 /* Evalua expr; devuelve 1 y deja el valor en *out, o 0 y el error en err */
784 static int evaluate(const char *expr, double *out, char *err, size_t errn)
785 {
786     parser_t p = { expr, expr, 0, "" };
787     double v = p_or(&p);
788     p_ws(&p);
789     if (!p.err && *p.s) p_fail(&p, "texto sobrante al final");
790     if (!p.err && !isfinite(v)) p_fail(&p, "el resultado no es un numero finito");
791     if (p.err) { snprintf(err, errn, "%s", p.emsg); return 0; }
792     *out = v;
793     return 1;
794 }
```

5 Caso C: ejecución de programas externos

Una solicitud que empieza con `run` (o con `!`) también la atiende un solo Meeseeks. Este crea a su vez un proceso nieto con `fork()` que reemplaza su imagen con `execl("/bin/sh", "sh", "-c", comando)`. Usar el intérprete permite pasar argumentos, tuberías y redirecciones (`run ls /usr/bin > /dev/null`). La entrada estándar del programa se redirige a `/dev/null` para que no consuma las solicitudes destinadas a la Box. El Meeseeks mide el tiempo y espera con `waitpid()`; la tarea se considera completada solo si el programa terminó normalmente (`WIFEXITED`) con código 0. Cualquier otro código (1, 3, 126, 127, ...) o una terminación por señal se reporta como no completada, indicando la causa.

Código 15: Ejecución de un programa externo (`src/meeseeks.c`, línea 825)

```
825 /* ---------------------------------------------------------------------- */
826 /*  C. Programas externos                                                 */
827 /*  El Meeseeks crea un proceso nieto con fork() que hace exec del comando */
828 /*  a traves de /bin/sh -c (permite argumentos, tuberias y redirecciones), */
829 /*  mide su tiempo de ejecucion y espera su valor de retorno con waitpid(). */
```

```c
/*  Exito <=> el programa termino normalmente con codigo 0.                   */
/* -------------------------------------------------------------------------- */
static void handle_run(int out_fd, msg_t *t)
{
    msg_t r;
    memset(&r, 0, sizeof r);
    r.type = M_RESULT; r.kind = K_RUN;
    r.solver = ME; r.solver.pid = getpid(); r.solver.ppid = getppid();

    say("Recibi por mi pipe la orden de ejecutar: %s. Ooooh, a la orden!", t->text);
    fflush(NULL);
    double t0 = now_s();
    pid_t c = fork();
    if (c < 0) {
        r.ok = 0;
        snprintf(r.text, sizeof r.text, "error: fork fallo (%s)", strerror(errno));
        write_msg(out_fd, &r);
        return;
    }
    if (c == 0) {
        int dn = open("/dev/null", O_RDONLY);   /* el programa no lee de la Box */
        if (dn >= 0) { dup2(dn, STDIN_FILENO); close(dn); }
        close(out_fd);
        execl("/bin/sh", "sh", "-c", t->text, (char *)NULL);
        _exit(127);                             /* exec fallo */
    }
    say("Programa externo lanzado como pid %d; espero su valor de retorno...", c);
    int st;
    while (waitpid(c, &st, 0) < 0 && errno == EINTR) ;
    double el = now_s() - t0;

    if (WIFEXITED(st)) {
        int code = WEXITSTATUS(st);
        r.ok = (code == 0);
        snprintf(r.text, sizeof r.text, "codigo de salida %d%s en %.4f s", code,
                 code == 127 ? " (comando no encontrado)" : code == 126 ? " (no ejecutable)" : ""
    , el);
    } else if (WIFSIGNALED(st)) {
        r.ok = 0;
        snprintf(r.text, sizeof r.text, "terminado por la senal %d (%s) en %.4f s",
                 WTERMSIG(st), strsignal(WTERMSIG(st)), el);
    }
    if (r.ok)
        say("El programa termino con exito (%s). Tarea completada! Gracias por dejarme servirle.
    Adios! *puff*", r.text);
    else
        say("El programa NO termino con exito (%s). La existencia es dolor! *puff*", r.text);
    write_msg(out_fd, &r);
}
```

6 Tiempo máximo, caos planetario y medición de tiempos

6.1 Caos planetario

La Box espera el resultado de cada solicitud con `poll()` sobre el pipe de subida de la raíz, con un tiempo límite igual al tiempo que falta para el máximo configurado (`-T`, 300 s por defecto, es decir 5 minutos). Si vence, se decreta el caos planetario: la Box envía `SIGTERM` y luego `SIGKILL` a todo el grupo de procesos de la solicitud. Esto es posible porque la raíz crea su propio grupo con `setpgid(0,0)` y todos sus descendientes lo heredan, de modo que un solo `kill(-pgid, ...)` alcanza a todo el árbol, incluidos los programas externos. Además, la Box se declara *subreaper* (`prctl(PR_SET_CHILD_SUBREAPER)`): los procesos cuyo padre murió en el caos quedan como hijos de la Box en lugar de `init`, y la Box los recoge con `waitpid(-1, …)` para no dejar zombis. Después imprime el informe y termina, como pide el enunciado. Con `-k` puede seguir aceptando solicitudes, y con `-x` (modo experimento) anuncia el caos pero deja que los Meeseeks continúen, para medir cuánto tiempo adicional necesitan.

Código 16: La Box atendiendo una solicitud (`src/meeseeks.c`, línea 982)

```
982 /* Atiende UNA solicitud: crea al Meeseeks raiz y espera su resultado.
983  * Devuelve 1 si se decreto caos planetario y la Box debe cerrarse.       */
984 static int run_task(int kind, const char *text, double d, double dur)
985 {
986     task_rec_t *R = &TASKS[NT < MAX_TASKS ? NT++ : MAX_TASKS - 1];
987     memset(R, 0, sizeof *R);
988     R->id = NT; R->kind = kind; R->chaos_at = -1.0;
989     snprintf(R->text, sizeof R->text, "%s", text);
990     R->d = d; R->d_random = (kind == K_TEXT && d < 0); R->dur_user = dur;
991
992     shared_reset();
993     sem_wait(&SH->slots);              /* la raiz ocupa un slot         */
994     SH->created = 1;
995
996     int down[2], up[2];
997     if (pipe(down) < 0 || pipe(up) < 0) { perror("pipe"); return 0; }
998
999     struct rusage r0, r1;
1000    getrusage(RUSAGE_CHILDREN, &r0);
1001    double t0 = now_s();
1002
1003    fflush(NULL);
1004    pid_t pid = fork();
1005    if (pid < 0) { perror("fork"); return 0; }
1006    if (pid == 0) {
1007        setpgid(0, 0);                 /* grupo propio: facilita el caos */
1008        signal(SIGINT, SIG_IGN);
1009        close(down[1]); close(up[0]);
1010        if (CFG.csv) { /* el hijo no debe tocar el FILE* de la Box */ }
1011        meeseeks_run(down[0], up[1], 1, 1);
1012        _exit(0);
1013    }
1014    setpgid(pid, pid);
1015    close(down[0]); close(up[1]);
1016
1017    say("Boton presionado: Mr. Meeseeks raiz (pid %d) atendera la solicitud #%d [%s] \"%s\"",
1018        pid, R->id, kind_name(kind), text);
```

```
1019
1020      msg_t m;
1021      memset(&m, 0, sizeof m);
1022      m.type = M_TASK; m.kind = kind; m.difficulty = d; m.duration = dur;
1023      snprintf(m.text, sizeof m.text, "%s", text);
1024      write_msg(down[1], &m);            /* mensaje privado via pipe        */
1025
1026      msg_t res;
1027      int have = 0, chaos = 0, killed = 0;
1028      double t_res = 0;
1029      for (;;) {
1030          int to = -1;
1031          if (CFG.max_time > 0 && !chaos) {
1032              double rem = t0 + CFG.max_time - now_s();
1033              to = rem <= 0 ? 0 : (int)(rem * 1000.0) + 1;
1034          }
1035          struct pollfd p = { up[0], POLLIN, 0 };
1036          int r = poll(&p, 1, to);
1037          if (r < 0 && errno == EINTR) {
1038              if (INTERRUPTED) { killed = 1; break; }
1039              continue;
1040          }
1041          if (r == 0) {                                  /* vencio el tiempo */
1042              chaos = 1;
1043              R->chaos_at = now_s() - t0;
1044              say("*** CAOS PLANETARIO! *** La solicitud #%d supero el tiempo maximo de %.1f s con
      %ld Mr. Meeseeks creados.",
1045                  R->id, CFG.max_time, SH->created);
1046              if (!CFG.experiment) { killed = 1; break; }
1047              say("Modo experimento (-x): no se termina; se mide cuanto tiempo adicional necesitan
      los Meeseeks.");
1048              continue;
1049          }
1050          int k = read_msg(up[0], &res);
1051          t_res = now_s();
1052          have = (k == 1);
1053          break;
1054      }
1055
1056      if (killed) {
1057          say("Terminando el grupo de procesos %d (SIGTERM y luego SIGKILL)...", pid);
1058          kill(-pid, SIGTERM);
1059          usleep(300000);
1060          kill(-pid, SIGKILL);
1061          t_res = now_s();
1062      }
1063
1064      int st;
1065      while (waitpid(pid, &st, 0) < 0 && errno == EINTR) ;
1066      /* La Box es subreaper: recoge huerfanos que hayan quedado */
1067      if (killed) { while (waitpid(-1, NULL, 0) > 0 || errno == EINTR) ; }
1068      else        { while (waitpid(-1, NULL, WNOHANG) > 0) ; }
1069      getrusage(RUSAGE_CHILDREN, &r1);
1070
1071      R->wall = t_res - t0;
```

```
1072     R->cpu  = (r1.ru_utime.tv_sec - r0.ru_utime.tv_sec) + (r1.ru_utime.tv_usec - r0.ru_utime.
         tv_usec) / 1e6
1073             + (r1.ru_stime.tv_sec - r0.ru_stime.tv_sec) + (r1.ru_stime.tv_usec - r0.ru_stime.
         tv_usec) / 1e6;
1074     R->created      = SH->created;
1075     R->max_alive    = SH->max_alive;
1076     R->max_level    = SH->max_level;
1077     R->failed_forks = SH->failed_forks;
1078     R->capped       = SH->capped;
1079     R->sum_sleep    = SH->sum_sleep;
1080     R->sum_active   = SH->sum_active;
1081     if (kind == K_TEXT) R->d = SH->root_d;
1082
1083     if (have && (res.type == M_SOLVED || (res.type == M_RESULT && res.ok))) R->status = 1;
1084     else if (killed) R->status = 2;
1085     else R->status = 0;
1086
1087     if (have && res.type == M_SOLVED) { R->solver = res.solver; R->sim = res.path_sleep; }
1088     else R->sim = SH->max_path_sleep;
1089     if (have && res.type == M_RESULT) { snprintf(R->result, sizeof R->result, "%s", res.text); R
         ->solver = res.solver; }
1090     R->real = R->wall - R->sim;
1091     if (R->real < 0) R->real = 0;
1092
1093     close(down[1]); close(up[0]);
1094
1095     const char *est = R->status == 1 ? "COMPLETADA" : R->status == 2 ? "CAOS PLANETARIO" : "NO
         COMPLETADA";
1096     say("Solicitud #%d %s. Tiempo total %.4f s | espera simulada %.4f s | tiempo real %.4f s |
         CPU %.4f s",
1097         R->id, est, R->wall, R->sim, R->real, R->cpu);
1098     say("   Mr. Meeseeks creados: %ld | nivel maximo: %d | pico simultaneo: %ld%s",
1099         R->created, R->max_level, R->max_alive, R->capped ? " | se activo el tope de contingencia
         " : "");
1100     if (R->chaos_at >= 0 && R->status == 1)
1101         say("   Caos declarado a los %.1f s; los Meeseeks necesitaron %.3f s adicionales para
         cumplir.",
1102             R->chaos_at, R->wall - CFG.max_time);
1103     if (R->result[0]) say("   Resultado: %s", R->result);
1104     csv_row(R);
1105
1106     return killed && !CFG.keep_going;
1107 }
```

6.2 Qué se mide

El enunciado pide tomar los tiempos de ejecución reales sin contar las esperas aleatorias de la simulación. Como los Meeseeks trabajan en paralelo, restar la suma de todas las esperas no tiene sentido (puede superar el tiempo total). Lo correcto es restar la espera del *camino crítico*: cada mensaje `M_TASK` lleva la espera simulada acumulada desde la raíz (`path_sleep`) y el ganador la devuelve en `M_SOLVED`. Para cada solicitud se registran:

- T_{total}: tiempo de pared desde que la Box crea la raíz hasta que recibe el resultado, medido con

`clock_gettime(CLOCK_MONOTONIC)`.

- T_{sim}: espera simulada a lo largo del camino raíz → ganador (o del camino más largo si nadie ganó).
- $T_{\text{real}} = T_{\text{total}} - T_{\text{sim}}$: el tiempo real que consumió el sistema operativo y el programa (crear procesos, copiar páginas, pasar mensajes, planificar, sincronizar). Para los casos B y C, $T_{\text{sim}} = 0$.
- CPU: tiempo de usuario más sistema de todos los descendientes, con `getrusage(RUSAGE_CHILDREN)`.
- Meeseeks creados, nivel máximo y pico de procesos vivos simultáneos.

Con la opción `-o` la Box agrega una fila CSV por solicitud; los experimentos de la sección 10 se construyen a partir de esos archivos.

7 Uso de Git

El desarrollo se hizo en un repositorio Git desde el primer archivo. Cada etapa quedó en su propio commit, incluida la corrección del interbloqueo descrita en la sección 3.6. El historial es:

```
1 \input{gitlog.txt}
```

8 Compilación y uso

```
1 make                    # gcc -Wall -Wextra -O2 -std=gnu11 ... -pthread -lm
2 ./meeseeks              # configuracion por defecto: T = 300 s, 3000 Meeseeks, 128 slots
3 ./meeseeks -T 60 -q -o datos.csv
```

Opción	Significado	Defecto
`-T`	tiempo máximo por solicitud antes del caos (0 = sin límite)	300 s
`-m`	máximo de Meeseeks por solicitud (contingencia)	3000
`-a`	Meeseeks trabajando a la vez (semáforo contador)	128
`-g`	ganancia de dilución	0.5
`-s`	semilla aleatoria	tiempo
`-q`	silencioso: solo mensajes de la Box	
`-x`	modo experimento: no terminar al vencer `-T`	
`-k`	seguir aceptando solicitudes después de un caos	
`-o`	agregar una fila CSV por solicitud	

Dentro de la Box, una línea de texto libre es una solicitud del caso A (la Box pregunta dificultad y duración; Enter deja ambas al azar), `calc <expresión>` es del caso B, `run <comando>` del caso C, y `adios` cierra la Box con el informe final.

9 Corridas de ejemplo

Se muestran tres corridas de cada caso, con la salida tal como la produce el programa (las solicitudes se alimentaron por la entrada estándar, por eso la Box repite cada línea leída). Las salidas completas

están en la carpeta `runs/` del repositorio; las dos más largas se recortaron aquí, conservando el inicio y el final.

9.1 Caso A con dificultad indicada por el usuario

Tres solicitudes con dificultades 95, 60 y 30 y duración aleatoria. La primera la resuelve la raíz sola ($d > 85$). En la segunda ($45 < d \leq 85$) la raíz falla su Tiro Meeseeks, invoca 2 ayudantes con la dificultad diluida (de 60 a 75.67 y 72.21) y uno de ellos acierta; la raíz detiene al resto del árbol e informa quién resolvió. En la tercera ($d \leq 45$) se invocan 5 ayudantes y el árbol llega al nivel 3. Se observan los mensajes de bienvenida con la tupla $(pid, ppid, N, i)$, los avisos por pipe, las despedidas recursivas y el informe final de la Box.

```
+------------------------------------------------------------+
|             BOX DE MR. MEESEEKS   (IC-6600, ITCR)          |
|  Escriba una solicitud, 'calc <expr>', 'run <comando>'     |
|  o despidase con 'adios'.                                  |
+------------------------------------------------------------+
Configuracion: tiempo maximo 300 s | tope 3000 Meeseeks | 128 trabajando a la vez | ganancia 0.50

Box> Quiero un cafe frances
  Dificultad 0-100 (Enter = a la suerte del Meeseeks): 95
  Duracion simulada en s (Enter = aleatoria 0.5-5):
[Box    pid 22067 | ppid 22065] Boton presionado: Mr. Meeseeks raiz (pid 22068) atendera la solicitud #1 [texto] "Quiero
   un cafe frances"
[pid 22068 | ppid 22067 | N=1 i=1] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22068, 22067, 1, 1)
[pid 22068 | ppid 22067 | N=1 i=1] Recibi por mi pipe la solicitud "Quiero un cafe frances" (dificultad 95.00). Ooooh,
   puedo hacerlo! Trabajando 1.12 s...
[pid 22068 | ppid 22067 | N=1 i=1] Solicitud completada: "Quiero un cafe frances"! Gracias por dejarme servirle. Adios! *
   puff*
[pid 22068 | ppid 22067 | N=1 i=1] INFORME FINAL: la solicitud "Quiero un cafe frances" fue resuelta por Mr. Meeseeks (pid
    22068, ppid 22067, N=1, i=1).
[Box    pid 22067 | ppid 22065] Solicitud #1 COMPLETADA. Tiempo total 1.1208 s | espera simulada 1.1180 s | tiempo real
   0.0027 s | CPU 0.0005 s
[Box    pid 22067 | ppid 22065]    Mr. Meeseeks creados: 1 | nivel maximo: 1 | pico simultaneo: 1

Box> Quiero ser feliz
  Dificultad 0-100 (Enter = a la suerte del Meeseeks): 60
  Duracion simulada en s (Enter = aleatoria 0.5-5):
[Box    pid 22067 | ppid 22065] Boton presionado: Mr. Meeseeks raiz (pid 22070) atendera la solicitud #2 [texto] "Quiero
   ser feliz"
[pid 22070 | ppid 22067 | N=1 i=1] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22070, 22067, 1, 1)
[pid 22070 | ppid 22067 | N=1 i=1] Recibi por mi pipe la solicitud "Quiero ser feliz" (dificultad 60.00). Ooooh, puedo
   hacerlo! Trabajando 1.49 s...
[pid 22070 | ppid 22067 | N=1 i=1] Esto es mas dificil de lo que pense (dificultad 60.00). Necesito ayuda! Invoco 2 Mr.
   Meeseeks (nivel 2)...
[pid 22071 | ppid 22070 | N=2 i=1] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22071, 22070, 2, 1)
[pid 22071 | ppid 22070 | N=2 i=1] Recibi por mi pipe la solicitud "Quiero ser feliz" (dificultad 75.66). Ooooh, puedo
   hacerlo! Trabajando 2.36 s...
[pid 22072 | ppid 22070 | N=2 i=2] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22072, 22070, 2, 2)
[pid 22072 | ppid 22070 | N=2 i=2] Recibi por mi pipe la solicitud "Quiero ser feliz" (dificultad 83.28). Ooooh, puedo
   hacerlo! Trabajando 2.73 s...
[pid 22071 | ppid 22070 | N=2 i=1] Tiro Meeseeks exitoso! (probabilidad 0.4330, saque 0.4251). Lo logre sin pedir ayuda.
[pid 22071 | ppid 22070 | N=2 i=1] Solicitud completada: "Quiero ser feliz"! Gracias por dejarme servirle. Adios! *puff*
[pid 22070 | ppid 22067 | N=1 i=1] Mi ayudante reporta que Mr. Meeseeks (pid 22071, ppid 22070, N=2, i=1) resolvio la
   solicitud! Aviso a mis hijos para que desaparezcan.
[pid 22072 | ppid 22070 | N=2 i=2] Me avisan por el pipe que la solicitud ya fue cumplida. Adios! *puff*
[pid 22070 | ppid 22067 | N=1 i=1] Mi trabajo aqui termino. Gracias por permitirme servirle. Adios! *puff*
[pid 22070 | ppid 22067 | N=1 i=1] INFORME FINAL: la solicitud "Quiero ser feliz" fue resuelta por Mr. Meeseeks (pid
   22071, ppid 22070, N=2, i=1).
[Box    pid 22067 | ppid 22065] Solicitud #2 COMPLETADA. Tiempo total 3.8558 s | espera simulada 3.8511 s | tiempo real
   0.0046 s | CPU 0.0018 s
[Box    pid 22067 | ppid 22065]    Mr. Meeseeks creados: 3 | nivel maximo: 2 | pico simultaneo: 3

Box> Quiero mejorar mi swing de golf
  Dificultad 0-100 (Enter = a la suerte del Meeseeks): 30
```

```
  Duracion simulada en s (Enter = aleatoria 0.5-5):
[Box    pid 22067 | ppid 22065] Boton presionado: Mr. Meeseeks raiz (pid 22074) atendera la solicitud #3 [texto] "Quiero mejorar mi swing de golf"
[pid 22074 | ppid 22067 | N=1 i=1] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22074, 22067, 1, 1)
[pid 22074 | ppid 22067 | N=1 i=1] Recibi por mi pipe la solicitud "Quiero mejorar mi swing de golf" (dificultad 30.00). Ooooh, puedo hacerlo! Trabajando 4.68 s...
[pid 22074 | ppid 22067 | N=1 i=1] Esto es mas dificil de lo que pense (dificultad 30.00). Necesito ayuda! Invoco 5 Mr. Meeseeks (nivel 2)...
[pid 22079 | ppid 22074 | N=2 i=1] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22079, 22074, 2, 1)
[pid 22079 | ppid 22074 | N=2 i=1] Recibi por mi pipe la solicitud "Quiero mejorar mi swing de golf" (dificultad 50.49). Ooooh, puedo hacerlo! Trabajando 1.50 s...
[pid 22080 | ppid 22074 | N=2 i=2] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22080, 22074, 2, 2)
[pid 22080 | ppid 22074 | N=2 i=2] Recibi por mi pipe la solicitud "Quiero mejorar mi swing de golf" (dificultad 54.32). Ooooh, puedo hacerlo! Trabajando 2.63 s...
[pid 22081 | ppid 22074 | N=2 i=3] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22081, 22074, 2, 3)
[pid 22081 | ppid 22074 | N=2 i=3] Recibi por mi pipe la solicitud "Quiero mejorar mi swing de golf" (dificultad 53.38). Ooooh, puedo hacerlo! Trabajando 2.34 s...
[pid 22083 | ppid 22074 | N=2 i=5] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22083, 22074, 2, 5)
[pid 22083 | ppid 22074 | N=2 i=5] Recibi por mi pipe la solicitud "Quiero mejorar mi swing de golf" (dificultad 50.09). Ooooh, puedo hacerlo! Trabajando 0.67 s...
[pid 22082 | ppid 22074 | N=2 i=4] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22082, 22074, 2, 4)
[pid 22082 | ppid 22074 | N=2 i=4] Recibi por mi pipe la solicitud "Quiero mejorar mi swing de golf" (dificultad 50.77). Ooooh, puedo hacerlo! Trabajando 1.65 s...
[pid 22083 | ppid 22074 | N=2 i=5] Esto es mas dificil de lo que pense (dificultad 50.09). Necesito ayuda! Invoco 2 Mr. Meeseeks (nivel 3)...
[pid 22085 | ppid 22083 | N=3 i=2] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22085, 22083, 3, 2)
[pid 22085 | ppid 22083 | N=3 i=2] Recibi por mi pipe la solicitud "Quiero mejorar mi swing de golf" (dificultad 72.37). Ooooh, puedo hacerlo! Trabajando 3.89 s...
[pid 22084 | ppid 22083 | N=3 i=1] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22084, 22083, 3, 1)
[pid 22084 | ppid 22083 | N=3 i=1] Recibi por mi pipe la solicitud "Quiero mejorar mi swing de golf" (dificultad 61.37). Ooooh, puedo hacerlo! Trabajando 1.13 s...
[pid 22079 | ppid 22074 | N=2 i=1] Esto es mas dificil de lo que pense (dificultad 50.49). Necesito ayuda! Invoco 2 Mr. Meeseeks (nivel 3)...
[pid 22086 | ppid 22079 | N=3 i=1] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22086, 22079, 3, 1)
[pid 22086 | ppid 22079 | N=3 i=1] Recibi por mi pipe la solicitud "Quiero mejorar mi swing de golf" (dificultad 64.69). Ooooh, puedo hacerlo! Trabajando 3.37 s...
[pid 22087 | ppid 22079 | N=3 i=2] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22087, 22079, 3, 2)
[pid 22087 | ppid 22079 | N=3 i=2] Recibi por mi pipe la solicitud "Quiero mejorar mi swing de golf" (dificultad 61.30). Ooooh, puedo hacerlo! Trabajando 3.91 s...
[pid 22082 | ppid 22074 | N=2 i=4] Esto es mas dificil de lo que pense (dificultad 50.77). Necesito ayuda! Invoco 2 Mr. Meeseeks (nivel 3)...
[pid 22088 | ppid 22082 | N=3 i=1] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22088, 22082, 3, 1)
[pid 22088 | ppid 22082 | N=3 i=1] Recibi por mi pipe la solicitud "Quiero mejorar mi swing de golf" (dificultad 73.58). Ooooh, puedo hacerlo! Trabajando 2.12 s...
[pid 22089 | ppid 22082 | N=3 i=2] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22089, 22082, 3, 2)
[pid 22089 | ppid 22082 | N=3 i=2] Recibi por mi pipe la solicitud "Quiero mejorar mi swing de golf" (dificultad 69.80). Ooooh, puedo hacerlo! Trabajando 0.95 s...
[pid 22084 | ppid 22083 | N=3 i=1] Esto es mas dificil de lo que pense (dificultad 61.37). Necesito ayuda! Invoco 2 Mr. Meeseeks (nivel 4)...
[pid 22091 | ppid 22084 | N=4 i=2] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22091, 22084, 4, 2)
[pid 22090 | ppid 22084 | N=4 i=1] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22090, 22084, 4, 1)
[pid 22091 | ppid 22084 | N=4 i=2] Recibi por mi pipe la solicitud "Quiero mejorar mi swing de golf" (dificultad 84.10). Ooooh, puedo hacerlo! Trabajando 5.72 s...
[pid 22090 | ppid 22084 | N=4 i=1] Recibi por mi pipe la solicitud "Quiero mejorar mi swing de golf" (dificultad 71.58). Ooooh, puedo hacerlo! Trabajando 2.90 s...
[pid 22081 | ppid 22074 | N=2 i=3] Tiro Meeseeks exitoso! (probabilidad 0.1521, saque 0.0490). Lo logre sin pedir ayuda.
[pid 22081 | ppid 22074 | N=2 i=3] Solicitud completada: "Quiero mejorar mi swing de golf"! Gracias por dejarme servirle. Adios! *puff*
[pid 22074 | ppid 22067 | N=1 i=1] Mi ayudante reporta que Mr. Meeseeks (pid 22081, ppid 22074, N=2, i=3) resolvio la solicitud! Aviso a mis hijos para que desaparezcan.
[pid 22080 | ppid 22074 | N=2 i=2] Me avisan por el pipe que la solicitud ya fue cumplida. Adios! *puff*
[pid 22086 | ppid 22079 | N=3 i=1] Me avisan por el pipe que la solicitud ya fue cumplida. Adios! *puff*
[pid 22087 | ppid 22079 | N=3 i=2] Me avisan por el pipe que la solicitud ya fue cumplida. Adios! *puff*
[pid 22088 | ppid 22082 | N=3 i=1] Me avisan por el pipe que la solicitud ya fue cumplida. Adios! *puff*
[pid 22079 | ppid 22074 | N=2 i=1] Me avisan por el pipe que la solicitud ya fue cumplida. Adios! *puff*
[pid 22089 | ppid 22082 | N=3 i=2] Me avisan por el pipe que la solicitud ya fue cumplida. Adios! *puff*
[pid 22085 | ppid 22083 | N=3 i=2] Me avisan por el pipe que la solicitud ya fue cumplida. Adios! *puff*
[pid 22082 | ppid 22074 | N=2 i=4] Me avisan por el pipe que la solicitud ya fue cumplida. Adios! *puff*
[pid 22090 | ppid 22084 | N=4 i=1] Me avisan por el pipe que la solicitud ya fue cumplida. Adios! *puff*
```

```
[pid 22091 | ppid 22084 | N=4 i=2] Me avisan por el pipe que la solicitud ya fue cumplida. Adios! *puff*
[pid 22084 | ppid 22083 | N=3 i=1] Me avisan por el pipe que la solicitud ya fue cumplida. Adios! *puff*
[pid 22083 | ppid 22074 | N=2 i=5] Me avisan por el pipe que la solicitud ya fue cumplida. Adios! *puff*
[pid 22074 | ppid 22067 | N=1 i=1] Mi trabajo aqui termino. Gracias por permitirme servirle. Adios! *puff*
[pid 22074 | ppid 22067 | N=1 i=1] INFORME FINAL: la solicitud "Quiero mejorar mi swing de golf" fue resuelta por Mr.
   Meeseeks (pid 22081, ppid 22074, N=2, i=3).
[Box    pid 22067 | ppid 22065] Solicitud #3 COMPLETADA. Tiempo total 7.0323 s | espera simulada 7.0207 s | tiempo real
   0.0116 s | CPU 0.0088 s
[Box    pid 22067 | ppid 22065]    Mr. Meeseeks creados: 14 | nivel maximo: 4 | pico simultaneo: 14

Box> adios
[Box    pid 22067 | ppid 22065] El usuario se despide. Cerrando la Box...

================================== INFORME DE LA BOX ==================================
#   Tipo  Solicitud                    Dific Estado          T.total    T.real      CPU Meeseeks  Niv  Resolvio (pid,ppid
   ,N,i)
1   texto Quiero un cafe frances       95.00 completada      1.1208s   0.0027s  0.0005s        1    1  (22068, 22067, 1,
   1)
2   texto Quiero ser feliz             60.00 completada      3.8558s   0.0046s  0.0018s        3    2  (22071, 22070, 2,
   1)
3   texto Quiero mejorar mi swing de g  30.00 completada      7.0323s   0.0116s  0.0088s       14    4  (22081, 22074, 2,
   3)
-------------------------------------------------------------------------------------------
Solicitudes: 3 | completadas: 3 | no completadas: 0 | caos: 0   (* dificultad a la suerte)
T.real = tiempo total menos la espera simulada del camino critico.
===========================================================================================
[Box    pid 22067 | ppid 22065] Gracias por usar la Box de Mr. Meeseeks. Hasta luego!
```

9.2 Caso A con la dificultad a la suerte del Meeseeks

El usuario presiona Enter en ambas preguntas: la raíz estima su dificultad con la distribución Beta$(4, 2)$ y la duración es aleatoria. En el informe final de la Box, el asterisco junto a la dificultad indica que fue elegida al azar.

```
+-------------------------------------------------------------+
|             BOX DE MR. MEESEEKS   (IC-6600, ITCR)           |
|  Escriba una solicitud, 'calc <expr>', 'run <comando>'      |
|  o despidase con 'adios'.                                   |
+-------------------------------------------------------------+
Configuracion: tiempo maximo 300 s | tope 3000 Meeseeks | 128 trabajando a la vez | ganancia 0.50

Box> Quiero aprender a tocar guitarra
  Dificultad 0-100 (Enter = a la suerte del Meeseeks):
  Duracion simulada en s (Enter = aleatoria 0.5-5):
[Box    pid 22093 | ppid 22065] Boton presionado: Mr. Meeseeks raiz (pid 22094) atendera la solicitud #1 [texto] "Quiero
   aprender a tocar guitarra"
[pid 22094 | ppid 22093 | N=1 i=1] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22094, 22093, 1, 1)
[pid 22094 | ppid 22093 | N=1 i=1] Nadie me dijo que tan dificil es... lo dejo a mi suerte (Beta(4,2)): dificultad 59.33
[pid 22094 | ppid 22093 | N=1 i=1] Recibi por mi pipe la solicitud "Quiero aprender a tocar guitarra" (dificultad 59.33).
   Ooooh, puedo hacerlo! Trabajando 1.46 s...
[pid 22094 | ppid 22093 | N=1 i=1] Esto es mas dificil de lo que pense (dificultad 59.33). Necesito ayuda! Invoco 2 Mr.
   Meeseeks (nivel 2)...
[pid 22099 | ppid 22094 | N=2 i=2] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22099, 22094, 2, 2)
[pid 22099 | ppid 22094 | N=2 i=2] Recibi por mi pipe la solicitud "Quiero aprender a tocar guitarra" (dificultad 81.50).
   Ooooh, puedo hacerlo! Trabajando 4.43 s...
[pid 22098 | ppid 22094 | N=2 i=1] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22098, 22094, 2, 1)
[pid 22098 | ppid 22094 | N=2 i=1] Recibi por mi pipe la solicitud "Quiero aprender a tocar guitarra" (dificultad 73.30).
   Ooooh, puedo hacerlo! Trabajando 2.69 s...
[pid 22098 | ppid 22094 | N=2 i=1] Esto es mas dificil de lo que pense (dificultad 73.30). Necesito ayuda! Invoco 2 Mr.
   Meeseeks (nivel 3)...
[pid 22103 | ppid 22098 | N=3 i=2] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22103, 22098, 3, 2)
[pid 22103 | ppid 22098 | N=3 i=2] Recibi por mi pipe la solicitud "Quiero aprender a tocar guitarra" (dificultad 88.83).
   Ooooh, puedo hacerlo! Trabajando 2.66 s...
[pid 22102 | ppid 22098 | N=3 i=1] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22102, 22098, 3, 1)
[pid 22102 | ppid 22098 | N=3 i=1] Recibi por mi pipe la solicitud "Quiero aprender a tocar guitarra" (dificultad 83.08).
   Ooooh, puedo hacerlo! Trabajando 4.96 s...
```

```
[pid 22099 | ppid 22094 | N=2 i=2] Tiro Meeseeks exitoso! (probabilidad 0.5414, saque 0.2975). Lo logre sin pedir ayuda.
[pid 22099 | ppid 22094 | N=2 i=2] Solicitud completada: "Quiero aprender a tocar guitarra"! Gracias por dejarme servirle. Adios! *puff*
[pid 22094 | ppid 22093 | N=1 i=1] Mi ayudante reporta que Mr. Meeseeks (pid 22099, ppid 22094, N=2, i=2) resolvio la solicitud! Aviso a mis hijos para que desaparezcan.
[pid 22102 | ppid 22098 | N=3 i=1] Me avisan por el pipe que la solicitud ya fue cumplida. Adios! *puff*
[pid 22103 | ppid 22098 | N=3 i=2] Me avisan por el pipe que la solicitud ya fue cumplida. Adios! *puff*
[pid 22098 | ppid 22094 | N=2 i=1] Me avisan por el pipe que la solicitud ya fue cumplida. Adios! *puff*
[pid 22094 | ppid 22093 | N=1 i=1] Mi trabajo aqui termino. Gracias por permitirme servirle. Adios! *puff*
[pid 22094 | ppid 22093 | N=1 i=1] INFORME FINAL: la solicitud "Quiero aprender a tocar guitarra" fue resuelta por Mr. Meeseeks (pid 22099, ppid 22094, N=2, i=2).
[Box    pid 22093 | ppid 22065] Solicitud #1 COMPLETADA. Tiempo total 5.9001 s | espera simulada 5.8926 s | tiempo real 0.0075 s | CPU 0.0032 s
[Box    pid 22093 | ppid 22065]    Mr. Meeseeks creados: 5 | nivel maximo: 3 | pico simultaneo: 5

Box> Quiero que mi planta no se muera
  Dificultad 0-100 (Enter = a la suerte del Meeseeks):
  Duracion simulada en s (Enter = aleatoria 0.5-5):
[Box    pid 22093 | ppid 22065] Boton presionado: Mr. Meeseeks raiz (pid 22107) atendera la solicitud #2 [texto] "Quiero que mi planta no se muera"
[pid 22107 | ppid 22093 | N=1 i=1] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22107, 22093, 1, 1)
[pid 22107 | ppid 22093 | N=1 i=1] Nadie me dijo que tan dificil es... lo dejo a mi suerte (Beta(4,2)): dificultad 94.19
[pid 22107 | ppid 22093 | N=1 i=1] Recibi por mi pipe la solicitud "Quiero que mi planta no se muera" (dificultad 94.19). Ooooh, puedo hacerlo! Trabajando 0.88 s...
[pid 22107 | ppid 22093 | N=1 i=1] Solicitud completada: "Quiero que mi planta no se muera"! Gracias por dejarme servirle. Adios! *puff*
[pid 22107 | ppid 22093 | N=1 i=1] INFORME FINAL: la solicitud "Quiero que mi planta no se muera" fue resuelta por Mr. Meeseeks (pid 22107, ppid 22093, N=1, i=1).
[Box    pid 22093 | ppid 22065] Solicitud #2 COMPLETADA. Tiempo total 0.8826 s | espera simulada 0.8805 s | tiempo real 0.0021 s | CPU 0.0004 s
[Box    pid 22093 | ppid 22065]    Mr. Meeseeks creados: 1 | nivel maximo: 1 | pico simultaneo: 1

Box> Quiero entender los punteros
  Dificultad 0-100 (Enter = a la suerte del Meeseeks):
  Duracion simulada en s (Enter = aleatoria 0.5-5):
[Box    pid 22093 | ppid 22065] Boton presionado: Mr. Meeseeks raiz (pid 22108) atendera la solicitud #3 [texto] "Quiero entender los punteros"
[pid 22108 | ppid 22093 | N=1 i=1] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22108, 22093, 1, 1)
[pid 22108 | ppid 22093 | N=1 i=1] Nadie me dijo que tan dificil es... lo dejo a mi suerte (Beta(4,2)): dificultad 46.74
[pid 22108 | ppid 22093 | N=1 i=1] Recibi por mi pipe la solicitud "Quiero entender los punteros" (dificultad 46.74). Ooooh, puedo hacerlo! Trabajando 3.75 s...
[pid 22108 | ppid 22093 | N=1 i=1] Esto es mas dificil de lo que pense (dificultad 46.74). Necesito ayuda! Invoco 2 Mr. Meeseeks (nivel 2)...
[pid 22116 | ppid 22108 | N=2 i=2] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22116, 22108, 2, 2)
[pid 22116 | ppid 22108 | N=2 i=2] Recibi por mi pipe la solicitud "Quiero entender los punteros" (dificultad 65.34). Ooooh, puedo hacerlo! Trabajando 3.70 s...
[pid 22115 | ppid 22108 | N=2 i=1] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22115, 22108, 2, 1)
[pid 22115 | ppid 22108 | N=2 i=1] Recibi por mi pipe la solicitud "Quiero entender los punteros" (dificultad 74.50). Ooooh, puedo hacerlo! Trabajando 2.97 s...
[pid 22115 | ppid 22108 | N=2 i=1] Tiro Meeseeks exitoso! (probabilidad 0.4135, saque 0.1257). Lo logre sin pedir ayuda.
[pid 22115 | ppid 22108 | N=2 i=1] Solicitud completada: "Quiero entender los punteros"! Gracias por dejarme servirle. Adios! *puff*
[pid 22108 | ppid 22093 | N=1 i=1] Mi ayudante reporta que Mr. Meeseeks (pid 22115, ppid 22108, N=2, i=1) resolvio la solicitud! Aviso a mis hijos para que desaparezcan.
[pid 22116 | ppid 22108 | N=2 i=2] Me avisan por el pipe que la solicitud ya fue cumplida. Adios! *puff*
[pid 22108 | ppid 22093 | N=1 i=1] Mi trabajo aqui termino. Gracias por permitirme servirle. Adios! *puff*
[pid 22108 | ppid 22093 | N=1 i=1] INFORME FINAL: la solicitud "Quiero entender los punteros" fue resuelta por Mr. Meeseeks (pid 22115, ppid 22108, N=2, i=1).
[Box    pid 22093 | ppid 22065] Solicitud #3 COMPLETADA. Tiempo total 6.7284 s | espera simulada 6.7190 s | tiempo real 0.0094 s | CPU 0.0018 s
[Box    pid 22093 | ppid 22065]    Mr. Meeseeks creados: 3 | nivel maximo: 2 | pico simultaneo: 3

Box> adios
[Box    pid 22093 | ppid 22065] El usuario se despide. Cerrando la Box...

================================== INFORME DE LA BOX ==================================
#   Tipo  Solicitud                    Dific Estado          T.total    T.real      CPU Meeseeks  Niv  Resolvio (pid,ppid,N,i)
1   texto Quiero aprender a tocar guit 59.33* completada      5.9001s   0.0075s  0.0032s        5    3  (22099, 22094, 2,
```

```
    2)
2   texto Quiero que mi planta no se m 94.19* completada      0.8826s   0.0021s  0.0004s        1    1  (22107, 22093, 1,
    1)
3   texto Quiero entender los punteros 46.74* completada      6.7284s   0.0094s  0.0018s        3    2  (22115, 22108, 2,
    1)
------------------------------------------------------------------------------------------
Solicitudes: 3 | completadas: 3 | no completadas: 0 | caos: 0   (* dificultad a la suerte)
T.real = tiempo total menos la espera simulada del camino critico.
==========================================================================================
[Box    pid 22093 | ppid 22065] Gracias por usar la Box de Mr. Meeseeks. Hasta luego!
```

9.3 Caso A imposible y plan de contingencia

Una solicitud con $d = 0$ y un tope reducido de 60 Meeseeks (`-m 60`) para que la salida sea legible. La dificultad no se diluye nunca, cada generación tarda más (factor de desesperación) y, al alcanzar el tope, las hojas reportan que no pueden invocar más ayudantes; la falla sube hasta la raíz y la solicitud se registra como no completada sin que el sistema colapse.

```
+------------------------------------------------------------+
|             BOX DE MR. MEESEEKS   (IC-6600, ITCR)          |
|  Escriba una solicitud, 'calc <expr>', 'run <comando>'     |
|  o despidase con 'adios'.                                  |
+------------------------------------------------------------+
Configuracion: tiempo maximo 300 s | tope 60 Meeseeks | 128 trabajando a la vez | ganancia 0.50

Box> Quiero que Jerry sea interesante
  Dificultad 0-100 (Enter = a la suerte del Meeseeks): 0
  Duracion simulada en s (Enter = aleatoria 0.5-5): 0.2
[Box    pid 22133 | ppid 22065] Advertencia: dificultad 0 = tarea imposible. Se aplicara el plan de contingencia.
[Box    pid 22133 | ppid 22065] Boton presionado: Mr. Meeseeks raiz (pid 22134) atendera la solicitud #1 [texto] "Quiero
    que Jerry sea interesante"
[pid 22134 | ppid 22133 | N=1 i=1] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22134, 22133, 1, 1)
[pid 22134 | ppid 22133 | N=1 i=1] Recibi por mi pipe la solicitud "Quiero que Jerry sea interesante" (dificultad 0.00).
    Ooooh, puedo hacerlo! Trabajando 0.20 s...
[pid 22134 | ppid 22133 | N=1 i=1] Esta tarea es IMPOSIBLE (dificultad 0.00)! La existencia es dolor! Invoco 5 Mr.
    Meeseeks mas (nivel 2)...
[pid 22135 | ppid 22134 | N=2 i=1] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22135, 22134, 2, 1)
[pid 22135 | ppid 22134 | N=2 i=1] Recibi por mi pipe la solicitud "Quiero que Jerry sea interesante" (dificultad 0.00).
    Ooooh, puedo hacerlo! Trabajando 0.30 s...
[pid 22137 | ppid 22134 | N=2 i=3] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22137, 22134, 2, 3)
[pid 22137 | ppid 22134 | N=2 i=3] Recibi por mi pipe la solicitud "Quiero que Jerry sea interesante" (dificultad 0.00).
    Ooooh, puedo hacerlo! Trabajando 0.30 s...
[pid 22139 | ppid 22134 | N=2 i=5] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22139, 22134, 2, 5)
[pid 22136 | ppid 22134 | N=2 i=2] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22136, 22134, 2, 2)
[pid 22139 | ppid 22134 | N=2 i=5] Recibi por mi pipe la solicitud "Quiero que Jerry sea interesante" (dificultad 0.00).
    Ooooh, puedo hacerlo! Trabajando 0.30 s...
[pid 22136 | ppid 22134 | N=2 i=2] Recibi por mi pipe la solicitud "Quiero que Jerry sea interesante" (dificultad 0.00).
    Ooooh, puedo hacerlo! Trabajando 0.30 s...
[pid 22138 | ppid 22134 | N=2 i=4] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22138, 22134, 2, 4)
[pid 22138 | ppid 22134 | N=2 i=4] Recibi por mi pipe la solicitud "Quiero que Jerry sea interesante" (dificultad 0.00).
    Ooooh, puedo hacerlo! Trabajando 0.30 s...
[pid 22135 | ppid 22134 | N=2 i=1] Esta tarea es IMPOSIBLE (dificultad 0.00)! La existencia es dolor! Invoco 5 Mr.
    Meeseeks mas (nivel 3)...
[pid 22138 | ppid 22134 | N=2 i=4] Esta tarea es IMPOSIBLE (dificultad 0.00)! La existencia es dolor! Invoco 5 Mr.
    Meeseeks mas (nivel 3)...
[pid 22140 | ppid 22135 | N=3 i=1] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22140, 22135, 3, 1)
[pid 22140 | ppid 22135 | N=3 i=1] Recibi por mi pipe la solicitud "Quiero que Jerry sea interesante" (dificultad 0.00).
    Ooooh, puedo hacerlo! Trabajando 0.40 s...
[pid 22141 | ppid 22135 | N=3 i=2] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22141, 22135, 3, 2)
[pid 22141 | ppid 22135 | N=3 i=2] Recibi por mi pipe la solicitud "Quiero que Jerry sea interesante" (dificultad 0.00).
    Ooooh, puedo hacerlo! Trabajando 0.40 s...
[pid 22143 | ppid 22135 | N=3 i=3] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22143, 22135, 3, 3)
[pid 22143 | ppid 22135 | N=3 i=3] Recibi por mi pipe la solicitud "Quiero que Jerry sea interesante" (dificultad 0.00).
    Ooooh, puedo hacerlo! Trabajando 0.40 s...
[pid 22142 | ppid 22138 | N=3 i=1] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22142, 22138, 3, 1)
```

```
[pid 22142 | ppid 22138 | N=3 i=1] Recibi por mi pipe la solicitud "Quiero que Jerry sea interesante" (dificultad 0.00). Ooooh, puedo hacerlo! Trabajando 0.40 s...
[pid 22144 | ppid 22135 | N=3 i=4] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22144, 22135, 3, 4)
[pid 22144 | ppid 22135 | N=3 i=4] Recibi por mi pipe la solicitud "Quiero que Jerry sea interesante" (dificultad 0.00). Ooooh, puedo hacerlo! Trabajando 0.40 s...
[pid 22145 | ppid 22138 | N=3 i=2] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22145, 22138, 3, 2)
[pid 22145 | ppid 22138 | N=3 i=2] Recibi por mi pipe la solicitud "Quiero que Jerry sea interesante" (dificultad 0.00). Ooooh, puedo hacerlo! Trabajando 0.40 s...
[pid 22147 | ppid 22138 | N=3 i=3] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22147, 22138, 3, 3)
[pid 22147 | ppid 22138 | N=3 i=3] Recibi por mi pipe la solicitud "Quiero que Jerry sea interesante" (dificultad 0.00). Ooooh, puedo hacerlo! Trabajando 0.40 s...
[pid 22146 | ppid 22135 | N=3 i=5] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22146, 22135, 3, 5)
[pid 22146 | ppid 22135 | N=3 i=5] Recibi por mi pipe la solicitud "Quiero que Jerry sea interesante" (dificultad 0.00). Ooooh, puedo hacerlo! Trabajando 0.40 s...
[pid 22148 | ppid 22138 | N=3 i=4] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22148, 22138, 3, 4)
[pid 22148 | ppid 22138 | N=3 i=4] Recibi por mi pipe la solicitud "Quiero que Jerry sea interesante" (dificultad 0.00). Ooooh, puedo hacerlo! Trabajando 0.40 s...
      [... 231 lineas omitidas; salida completa en runs/caso_A_imposible.txt ...]
[pid 22182 | ppid 22146 | N=4 i=5] Contingencia: no puedo invocar mas Mr. Meeseeks (tope de 60 por solicitud o fork fallido).
[pid 22182 | ppid 22146 | N=4 i=5] Ninguno de mis ayudantes pudo con "Quiero que Jerry sea interesante". La existencia es dolor! Me rindo. *puff*
[pid 22194 | ppid 22148 | N=4 i=1] Esta tarea es IMPOSIBLE (dificultad 0.00)! La existencia es dolor! Invoco 5 Mr. Meeseeks mas (nivel 5)...
[pid 22194 | ppid 22148 | N=4 i=1] Contingencia: no puedo invocar mas Mr. Meeseeks (tope de 60 por solicitud o fork fallido).
[pid 22194 | ppid 22148 | N=4 i=1] Ninguno de mis ayudantes pudo con "Quiero que Jerry sea interesante". La existencia es dolor! Me rindo. *puff*
[pid 22185 | ppid 22159 | N=4 i=5] Esta tarea es IMPOSIBLE (dificultad 0.00)! La existencia es dolor! Invoco 5 Mr. Meeseeks mas (nivel 5)...
[pid 22185 | ppid 22159 | N=4 i=5] Contingencia: no puedo invocar mas Mr. Meeseeks (tope de 60 por solicitud o fork fallido).
[pid 22185 | ppid 22159 | N=4 i=5] Ninguno de mis ayudantes pudo con "Quiero que Jerry sea interesante". La existencia es dolor! Me rindo. *puff*
[pid 22146 | ppid 22135 | N=3 i=5] Ninguno de mis ayudantes pudo con "Quiero que Jerry sea interesante". La existencia es dolor! Me rindo. *puff*
[pid 22135 | ppid 22134 | N=2 i=1] Ninguno de mis ayudantes pudo con "Quiero que Jerry sea interesante". La existencia es dolor! Me rindo. *puff*
[pid 22184 | ppid 22159 | N=4 i=4] Esta tarea es IMPOSIBLE (dificultad 0.00)! La existencia es dolor! Invoco 5 Mr. Meeseeks mas (nivel 5)...
[pid 22184 | ppid 22159 | N=4 i=4] Contingencia: no puedo invocar mas Mr. Meeseeks (tope de 60 por solicitud o fork fallido).
[pid 22184 | ppid 22159 | N=4 i=4] Ninguno de mis ayudantes pudo con "Quiero que Jerry sea interesante". La existencia es dolor! Me rindo. *puff*
[pid 22191 | ppid 22149 | N=4 i=1] Esta tarea es IMPOSIBLE (dificultad 0.00)! La existencia es dolor! Invoco 5 Mr. Meeseeks mas (nivel 5)...
[pid 22191 | ppid 22149 | N=4 i=1] Contingencia: no puedo invocar mas Mr. Meeseeks (tope de 60 por solicitud o fork fallido).
[pid 22191 | ppid 22149 | N=4 i=1] Ninguno de mis ayudantes pudo con "Quiero que Jerry sea interesante". La existencia es dolor! Me rindo. *puff*
[pid 22192 | ppid 22149 | N=4 i=2] Esta tarea es IMPOSIBLE (dificultad 0.00)! La existencia es dolor! Invoco 5 Mr. Meeseeks mas (nivel 5)...
[pid 22192 | ppid 22149 | N=4 i=2] Contingencia: no puedo invocar mas Mr. Meeseeks (tope de 60 por solicitud o fork fallido).
[pid 22192 | ppid 22149 | N=4 i=2] Ninguno de mis ayudantes pudo con "Quiero que Jerry sea interesante". La existencia es dolor! Me rindo. *puff*
[pid 22149 | ppid 22138 | N=3 i=5] Ninguno de mis ayudantes pudo con "Quiero que Jerry sea interesante". La existencia es dolor! Me rindo. *puff*
[pid 22148 | ppid 22138 | N=3 i=4] Ninguno de mis ayudantes pudo con "Quiero que Jerry sea interesante". La existencia es dolor! Me rindo. *puff*
[pid 22159 | ppid 22139 | N=3 i=5] Ninguno de mis ayudantes pudo con "Quiero que Jerry sea interesante". La existencia es dolor! Me rindo. *puff*
[pid 22138 | ppid 22134 | N=2 i=4] Ninguno de mis ayudantes pudo con "Quiero que Jerry sea interesante". La existencia es dolor! Me rindo. *puff*
[pid 22139 | ppid 22134 | N=2 i=5] Ninguno de mis ayudantes pudo con "Quiero que Jerry sea interesante". La existencia es dolor! Me rindo. *puff*
[pid 22134 | ppid 22133 | N=1 i=1] Ninguno de mis ayudantes pudo con "Quiero que Jerry sea interesante". La existencia es dolor! Me rindo. *puff*
[pid 22134 | ppid 22133 | N=1 i=1] INFORME FINAL: la solicitud "Quiero que Jerry sea interesante" NO pudo completarse.
```

```
[Box    pid 22133 | ppid 22065] Solicitud #1 NO COMPLETADA. Tiempo total 1.4087 s | espera simulada 1.4000 s | tiempo real 0.0087 s | CPU 0.0219 s
[Box    pid 22133 | ppid 22065]    Mr. Meeseeks creados: 60 | nivel maximo: 4 | pico simultaneo: 58 | se activo el tope de contingencia

Box> adios
[Box    pid 22133 | ppid 22065] El usuario se despide. Cerrando la Box...

================================== INFORME DE LA BOX ==================================
#   Tipo  Solicitud                    Dific Estado          T.total    T.real      CPU Meeseeks  Niv  Resolvio (pid,ppid,N,i)
1   texto Quiero que Jerry sea interes   0.00 no completada   1.4087s   0.0087s  0.0219s       60    4  -
-------------------------------------------------------------------------------------
Solicitudes: 1 | completadas: 0 | no completadas: 1 | caos: 0   (* dificultad a la suerte)
T.real = tiempo total menos la espera simulada del camino critico.
=====================================================================================
[Box    pid 22133 | ppid 22065] Gracias por usar la Box de Mr. Meeseeks. Hasta luego!
```

9.4 Caso B: cálculos

Tres expresiones: una aritmética, una lógica y una con división entre cero. La segunda es interesante: $\sqrt{2} \cdot \sqrt{2}$ no es exactamente 2 en punto flotante (2.0000000000000004), así que la igualdad vale 0; aun así la expresión completa es verdadera por la disyunción. La tercera se reporta como error y como solicitud no completada.

```
+-------------------------------------------------------------+
|             BOX DE MR. MEESEEKS   (IC-6600, ITCR)           |
|  Escriba una solicitud, 'calc <expr>', 'run <comando>'      |
|  o despidase con 'adios'.                                   |
+-------------------------------------------------------------+
Configuracion: tiempo maximo 300 s | tope 3000 Meeseeks | 128 trabajando a la vez | ganancia 0.50

Box> calc (2+3)*4^2/8 - 7 % 3
[Box    pid 22039 | ppid 22029] Boton presionado: Mr. Meeseeks raiz (pid 22040) atendera la solicitud #1 [calc] "(2+3)*4^2/8 - 7 % 3"
[pid 22040 | ppid 22039 | N=1 i=1] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22040, 22039, 1, 1)
[pid 22040 | ppid 22039 | N=1 i=1] Recibi por mi pipe un calculo: "(2+3)*4^2/8 - 7 % 3". Ooooh, eso es facil!
[pid 22040 | ppid 22039 | N=1 i=1] Resultado: (2+3)*4^2/8 - 7 % 3 = 9. Calculo completado! Gracias por dejarme servirle. Adios! *puff*
[Box    pid 22039 | ppid 22029] Solicitud #1 COMPLETADA. Tiempo total 0.0004 s | espera simulada 0.0000 s | tiempo real 0.0004 s | CPU 0.0002 s
[Box    pid 22039 | ppid 22029]    Mr. Meeseeks creados: 1 | nivel maximo: 1 | pico simultaneo: 1
[Box    pid 22039 | ppid 22029]    Resultado: (2+3)*4^2/8 - 7 % 3 = 9

Box> calc sqrt(2)*sqrt(2) == 2 || 1 > 0 && !false
[Box    pid 22039 | ppid 22029] Boton presionado: Mr. Meeseeks raiz (pid 22041) atendera la solicitud #2 [calc] "sqrt(2)*sqrt(2) == 2 || 1 > 0 && !false"
[pid 22041 | ppid 22039 | N=1 i=1] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22041, 22039, 1, 1)
[pid 22041 | ppid 22039 | N=1 i=1] Recibi por mi pipe un calculo: "sqrt(2)*sqrt(2) == 2 || 1 > 0 && !false". Ooooh, eso es facil!
[pid 22041 | ppid 22039 | N=1 i=1] Resultado: sqrt(2)*sqrt(2) == 2 || 1 > 0 && !false = 1. Calculo completado! Gracias por dejarme servirle. Adios! *puff*
[Box    pid 22039 | ppid 22029] Solicitud #2 COMPLETADA. Tiempo total 0.0003 s | espera simulada 0.0000 s | tiempo real 0.0003 s | CPU 0.0002 s
[Box    pid 22039 | ppid 22029]    Mr. Meeseeks creados: 1 | nivel maximo: 1 | pico simultaneo: 1
[Box    pid 22039 | ppid 22029]    Resultado: sqrt(2)*sqrt(2) == 2 || 1 > 0 && !false = 1

Box> calc 10 / (5 - 5)
[Box    pid 22039 | ppid 22029] Boton presionado: Mr. Meeseeks raiz (pid 22042) atendera la solicitud #3 [calc] "10 / (5 - 5)"
[pid 22042 | ppid 22039 | N=1 i=1] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22042, 22039, 1, 1)
[pid 22042 | ppid 22039 | N=1 i=1] Recibi por mi pipe un calculo: "10 / (5 - 5)". Ooooh, eso es facil!
[pid 22042 | ppid 22039 | N=1 i=1] No pude evaluar "10 / (5 - 5)": division entre cero (posicion 13). La existencia es dolor! *puff*
```

```
[Box    pid 22039 | ppid 22029] Solicitud #3 NO COMPLETADA. Tiempo total 0.0002 s | espera simulada 0.0000 s | tiempo real 0.0002 s | CPU 0.0002 s
[Box    pid 22039 | ppid 22029]    Mr. Meeseeks creados: 1 | nivel maximo: 1 | pico simultaneo: 1
[Box    pid 22039 | ppid 22029]    Resultado: error: division entre cero (posicion 13)

Box> adios
[Box    pid 22039 | ppid 22029] El usuario se despide. Cerrando la Box...

================================== INFORME DE LA BOX ==================================
#   Tipo  Solicitud                    Dific Estado          T.total    T.real      CPU Meeseeks  Niv  Resolvio (pid,ppid,N,i)
1   calc  (2+3)*4^2/8 - 7 % 3              - completada      0.0004s   0.0004s  0.0002s        1    1  (22040, 22039, 1, 1)
2   calc  sqrt(2)*sqrt(2) == 2 || 1 >      - completada      0.0003s   0.0003s  0.0002s        1    1  (22041, 22039, 1, 1)
3   calc  10 / (5 - 5)                     - no completada   0.0002s   0.0002s  0.0002s        1    1  -
--------------------------------------------------------------------------------------
Solicitudes: 3 | completadas: 2 | no completadas: 1 | caos: 0   (* dificultad a la suerte)
T.real = tiempo total menos la espera simulada del camino critico.
======================================================================================
[Box    pid 22039 | ppid 22029] Gracias por usar la Box de Mr. Meeseeks. Hasta luego!
```

9.5 Caso C: programas externos

Tres programas: `ls` (código 0), una tubería con `gcc --version` (código 0) y un comando inexistente, que el intérprete reporta con el código 127 y por lo tanto queda como no completado.

```
+------------------------------------------------------------+
|             BOX DE MR. MEESEEKS   (IC-6600, ITCR)          |
|  Escriba una solicitud, 'calc <expr>', 'run <comando>'     |
|  o despidase con 'adios'.                                  |
+------------------------------------------------------------+
Configuracion: tiempo maximo 300 s | tope 3000 Meeseeks | 128 trabajando a la vez | ganancia 0.50

Box> run ls -l src
[Box    pid 22044 | ppid 22029] Boton presionado: Mr. Meeseeks raiz (pid 22045) atendera la solicitud #1 [run] "ls -l src"
[pid 22045 | ppid 22044 | N=1 i=1] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22045, 22044, 1, 1)
[pid 22045 | ppid 22044 | N=1 i=1] Recibi por mi pipe la orden de ejecutar: ls -l src. Ooooh, a la orden!
[pid 22045 | ppid 22044 | N=1 i=1] Programa externo lanzado como pid 22046; espero su valor de retorno...
total 52
-rw-r--r-- 1 root root 51007 Oct 12 23:56 meeseeks.c
[pid 22045 | ppid 22044 | N=1 i=1] El programa termino con exito (codigo de salida 0 en 0.0039 s). Tarea completada! Gracias por dejarme servirle. Adios! *puff*
[Box    pid 22044 | ppid 22029] Solicitud #1 COMPLETADA. Tiempo total 0.0042 s | espera simulada 0.0000 s | tiempo real 0.0042 s | CPU 0.0043 s
[Box    pid 22044 | ppid 22029]    Mr. Meeseeks creados: 1 | nivel maximo: 1 | pico simultaneo: 1
[Box    pid 22044 | ppid 22029]    Resultado: codigo de salida 0 en 0.0039 s

Box> run gcc --version | head -1
[Box    pid 22044 | ppid 22029] Boton presionado: Mr. Meeseeks raiz (pid 22048) atendera la solicitud #2 [run] "gcc --version | head -1"
[pid 22048 | ppid 22044 | N=1 i=1] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22048, 22044, 1, 1)
[pid 22048 | ppid 22044 | N=1 i=1] Recibi por mi pipe la orden de ejecutar: gcc --version | head -1. Ooooh, a la orden!
[pid 22048 | ppid 22044 | N=1 i=1] Programa externo lanzado como pid 22049; espero su valor de retorno...
gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
[pid 22048 | ppid 22044 | N=1 i=1] El programa termino con exito (codigo de salida 0 en 0.0041 s). Tarea completada! Gracias por dejarme servirle. Adios! *puff*
[Box    pid 22044 | ppid 22029] Solicitud #2 COMPLETADA. Tiempo total 0.0043 s | espera simulada 0.0000 s | tiempo real 0.0043 s | CPU 0.0060 s
[Box    pid 22044 | ppid 22029]    Mr. Meeseeks creados: 1 | nivel maximo: 1 | pico simultaneo: 1
[Box    pid 22044 | ppid 22029]    Resultado: codigo de salida 0 en 0.0041 s

Box> run programa_inexistente
[Box    pid 22044 | ppid 22029] Boton presionado: Mr. Meeseeks raiz (pid 22052) atendera la solicitud #3 [run] "programa_inexistente"
[pid 22052 | ppid 22044 | N=1 i=1] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22052, 22044, 1, 1)
```

```
[pid 22052 | ppid 22044 | N=1 i=1] Recibi por mi pipe la orden de ejecutar: programa_inexistente. Ooooh, a la orden!
[pid 22052 | ppid 22044 | N=1 i=1] Programa externo lanzado como pid 22053; espero su valor de retorno...
sh: 1: programa_inexistente: not found
[pid 22052 | ppid 22044 | N=1 i=1] El programa NO termino con exito (codigo de salida 127 (comando no encontrado) en 0.0019 s). La existencia es dolor! *puff*
[Box    pid 22044 | ppid 22029] Solicitud #3 NO COMPLETADA. Tiempo total 0.0021 s | espera simulada 0.0000 s | tiempo real 0.0021 s | CPU 0.0020 s
[Box    pid 22044 | ppid 22029]    Mr. Meeseeks creados: 1 | nivel maximo: 1 | pico simultaneo: 1
[Box    pid 22044 | ppid 22029]    Resultado: codigo de salida 127 (comando no encontrado) en 0.0019 s

Box> adios
[Box    pid 22044 | ppid 22029] El usuario se despide. Cerrando la Box...

================================== INFORME DE LA BOX ==================================
#   Tipo  Solicitud                    Dific Estado          T.total    T.real      CPU Meeseeks  Niv  Resolvio (pid,ppid,N,i)
1   run   ls -l src                         - completada      0.0042s   0.0042s  0.0043s        1    1  (22045, 22044, 1, 1)
2   run   gcc --version | head -1           - completada      0.0043s   0.0043s  0.0060s        1    1  (22048, 22044, 1, 1)
3   run   programa_inexistente              - no completada   0.0021s   0.0021s  0.0020s        1    1  -
-------------------------------------------------------------------------------------
Solicitudes: 3 | completadas: 2 | no completadas: 1 | caos: 0   (* dificultad a la suerte)
T.real = tiempo total menos la espera simulada del camino critico.
=====================================================================================
[Box    pid 22044 | ppid 22029] Gracias por usar la Box de Mr. Meeseeks. Hasta luego!
```

9.6 Caos planetario

Una tarea imposible con el tope desactivado en la práctica (`-m 100000`) y un tiempo máximo reducido a 20 s para la demostración. Al vencer el plazo la Box decreta el caos, termina el grupo de procesos y cierra el programa con el informe. El experimento E6 estudia qué pasa si, en lugar de terminar, se deja seguir a los Meeseeks.

```
+-------------------------------------------------------------+
|             BOX DE MR. MEESEEKS   (IC-6600, ITCR)           |
|  Escriba una solicitud, 'calc <expr>', 'run <comando>'      |
|  o despidase con 'adios'.                                   |
+-------------------------------------------------------------+
Configuracion: tiempo maximo 20 s | tope 100000 Meeseeks | 128 trabajando a la vez | ganancia 0.50

Box> Quiero que Jerry sea interesante
  Dificultad 0-100 (Enter = a la suerte del Meeseeks): 0
  Duracion simulada en s (Enter = aleatoria 0.5-5):
[Box    pid 22196 | ppid 22065] Advertencia: dificultad 0 = tarea imposible. Se aplicara el plan de contingencia.
[Box    pid 22196 | ppid 22065] Boton presionado: Mr. Meeseeks raiz (pid 22197) atendera la solicitud #1 [texto] "Quiero que Jerry sea interesante"
[pid 22197 | ppid 22196 | N=1 i=1] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22197, 22196, 1, 1)
[pid 22197 | ppid 22196 | N=1 i=1] Recibi por mi pipe la solicitud "Quiero que Jerry sea interesante" (dificultad 0.00). Ooooh, puedo hacerlo! Trabajando 3.75 s...
[pid 22197 | ppid 22196 | N=1 i=1] Esta tarea es IMPOSIBLE (dificultad 0.00)! La existencia es dolor! Invoco 5 Mr. Meeseeks mas (nivel 2)...
[pid 22214 | ppid 22197 | N=2 i=1] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22214, 22197, 2, 1)
[pid 22214 | ppid 22197 | N=2 i=1] Recibi por mi pipe la solicitud "Quiero que Jerry sea interesante" (dificultad 0.00). Ooooh, puedo hacerlo! Trabajando 1.75 s...
[pid 22216 | ppid 22197 | N=2 i=3] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22216, 22197, 2, 3)
[pid 22216 | ppid 22197 | N=2 i=3] Recibi por mi pipe la solicitud "Quiero que Jerry sea interesante" (dificultad 0.00). Ooooh, puedo hacerlo! Trabajando 5.32 s...
[pid 22218 | ppid 22197 | N=2 i=5] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22218, 22197, 2, 5)
[pid 22218 | ppid 22197 | N=2 i=5] Recibi por mi pipe la solicitud "Quiero que Jerry sea interesante" (dificultad 0.00). Ooooh, puedo hacerlo! Trabajando 4.54 s...
[pid 22215 | ppid 22197 | N=2 i=2] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22215, 22197, 2, 2)
[pid 22215 | ppid 22197 | N=2 i=2] Recibi por mi pipe la solicitud "Quiero que Jerry sea interesante" (dificultad 0.00). Ooooh, puedo hacerlo! Trabajando 1.63 s...
[pid 22217 | ppid 22197 | N=2 i=4] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22217, 22197, 2, 4)
```

```
[pid 22217 | ppid 22197 | N=2 i=4] Recibi por mi pipe la solicitud "Quiero que Jerry sea interesante" (dificultad 0.00). Ooooh, puedo hacerlo! Trabajando 3.11 s...
[pid 22215 | ppid 22197 | N=2 i=2] Esta tarea es IMPOSIBLE (dificultad 0.00)! La existencia es dolor! Invoco 5 Mr. Meeseeks mas (nivel 3)...
[pid 22246 | ppid 22215 | N=3 i=1] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22246, 22215, 3, 1)
[pid 22246 | ppid 22215 | N=3 i=1] Recibi por mi pipe la solicitud "Quiero que Jerry sea interesante" (dificultad 0.00). Ooooh, puedo hacerlo! Trabajando 4.71 s...
[pid 22249 | ppid 22215 | N=3 i=4] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22249, 22215, 3, 4)
[pid 22249 | ppid 22215 | N=3 i=4] Recibi por mi pipe la solicitud "Quiero que Jerry sea interesante" (dificultad 0.00). Ooooh, puedo hacerlo! Trabajando 5.60 s...
   [... 599 lineas omitidas; salida completa en runs/caos_T20.txt ...]
[pid 22535 | ppid 22448 | N=7 i=1] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22535, 22448, 7, 1)
[pid 22535 | ppid 22448 | N=7 i=1] Recibi por mi pipe la solicitud "Quiero que Jerry sea interesante" (dificultad 0.00). Ooooh, puedo hacerlo! Trabajando 6.42 s...
[pid 22359 | ppid 22308 | N=4 i=3] Esta tarea es IMPOSIBLE (dificultad 0.00)! La existencia es dolor! Invoco 5 Mr. Meeseeks mas (nivel 5)...
[pid 22536 | ppid 22359 | N=5 i=1] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22536, 22359, 5, 1)
[pid 22536 | ppid 22359 | N=5 i=1] Recibi por mi pipe la solicitud "Quiero que Jerry sea interesante" (dificultad 0.00). Ooooh, puedo hacerlo! Trabajando 4.87 s...
[pid 22427 | ppid 22364 | N=6 i=1] Esta tarea es IMPOSIBLE (dificultad 0.00)! La existencia es dolor! Invoco 5 Mr. Meeseeks mas (nivel 7)...
[pid 22537 | ppid 22427 | N=7 i=1] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22537, 22427, 7, 1)
[pid 22537 | ppid 22427 | N=7 i=1] Recibi por mi pipe la solicitud "Quiero que Jerry sea interesante" (dificultad 0.00). Ooooh, puedo hacerlo! Trabajando 7.90 s...
[pid 22349 | ppid 22281 | N=4 i=3] Esta tarea es IMPOSIBLE (dificultad 0.00)! La existencia es dolor! Invoco 5 Mr. Meeseeks mas (nivel 5)...
[pid 22538 | ppid 22349 | N=5 i=1] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22538, 22349, 5, 1)
[pid 22538 | ppid 22349 | N=5 i=1] Recibi por mi pipe la solicitud "Quiero que Jerry sea interesante" (dificultad 0.00). Ooooh, puedo hacerlo! Trabajando 5.12 s...
[pid 22479 | ppid 22319 | N=4 i=1] Esta tarea es IMPOSIBLE (dificultad 0.00)! La existencia es dolor! Invoco 5 Mr. Meeseeks mas (nivel 5)...
[pid 22539 | ppid 22479 | N=5 i=1] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22539, 22479, 5, 1)
[pid 22539 | ppid 22479 | N=5 i=1] Recibi por mi pipe la solicitud "Quiero que Jerry sea interesante" (dificultad 0.00). Ooooh, puedo hacerlo! Trabajando 1.59 s...
[pid 22475 | ppid 22317 | N=4 i=1] Esta tarea es IMPOSIBLE (dificultad 0.00)! La existencia es dolor! Invoco 5 Mr. Meeseeks mas (nivel 5)...
[pid 22540 | ppid 22475 | N=5 i=1] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22540, 22475, 5, 1)
[pid 22540 | ppid 22475 | N=5 i=1] Recibi por mi pipe la solicitud "Quiero que Jerry sea interesante" (dificultad 0.00). Ooooh, puedo hacerlo! Trabajando 7.11 s...
[pid 22374 | ppid 22264 | N=5 i=3] Esta tarea es IMPOSIBLE (dificultad 0.00)! La existencia es dolor! Invoco 5 Mr. Meeseeks mas (nivel 6)...
[pid 22541 | ppid 22374 | N=6 i=1] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22541, 22374, 6, 1)
[pid 22541 | ppid 22374 | N=6 i=1] Recibi por mi pipe la solicitud "Quiero que Jerry sea interesante" (dificultad 0.00). Ooooh, puedo hacerlo! Trabajando 8.53 s...
[pid 22328 | ppid 22246 | N=4 i=2] Esta tarea es IMPOSIBLE (dificultad 0.00)! La existencia es dolor! Invoco 5 Mr. Meeseeks mas (nivel 5)...
[pid 22542 | ppid 22328 | N=5 i=1] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22542, 22328, 5, 1)
[pid 22542 | ppid 22328 | N=5 i=1] Recibi por mi pipe la solicitud "Quiero que Jerry sea interesante" (dificultad 0.00). Ooooh, puedo hacerlo! Trabajando 9.72 s...
[pid 22381 | ppid 22306 | N=4 i=5] Esta tarea es IMPOSIBLE (dificultad 0.00)! La existencia es dolor! Invoco 5 Mr. Meeseeks mas (nivel 5)...
[pid 22543 | ppid 22351 | N=5 i=2] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22543, 22351, 5, 2)
[pid 22543 | ppid 22351 | N=5 i=2] Recibi por mi pipe la solicitud "Quiero que Jerry sea interesante" (dificultad 0.00). Ooooh, puedo hacerlo! Trabajando 8.69 s...
[pid 22327 | ppid 22246 | N=4 i=1] Esta tarea es IMPOSIBLE (dificultad 0.00)! La existencia es dolor! Invoco 5 Mr. Meeseeks mas (nivel 5)...
[pid 22544 | ppid 22327 | N=5 i=1] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22544, 22327, 5, 1)
[pid 22544 | ppid 22327 | N=5 i=1] Recibi por mi pipe la solicitud "Quiero que Jerry sea interesante" (dificultad 0.00). Ooooh, puedo hacerlo! Trabajando 9.26 s...
[pid 22480 | ppid 22342 | N=6 i=1] Esta tarea es IMPOSIBLE (dificultad 0.00)! La existencia es dolor! Invoco 5 Mr. Meeseeks mas (nivel 7)...
[pid 22545 | ppid 22480 | N=7 i=1] Hi, I'm Mr. Meeseeks! Look at Meeeee. (22545, 22480, 7, 1)
[pid 22545 | ppid 22480 | N=7 i=1] Recibi por mi pipe la solicitud "Quiero que Jerry sea interesante" (dificultad 0.00). Ooooh, puedo hacerlo! Trabajando 18.75 s...
[Box    pid 22196 | ppid 22065] *** CAOS PLANETARIO! *** La solicitud #1 supero el tiempo maximo de 20.0 s con 259 Mr. Meeseeks creados.
[Box    pid 22196 | ppid 22065] Terminando el grupo de procesos 22197 (SIGTERM y luego SIGKILL)...
[Box    pid 22196 | ppid 22065] Solicitud #1 CAOS PLANETARIO. Tiempo total 20.3357 s | espera simulada 19.9503 s | tiempo real 0.3853 s | CPU 0.5063 s
```

```
[Box    pid 22196 | ppid 22065]    Mr. Meeseeks creados: 259 | nivel maximo: 7 | pico simultaneo: 259

================================== INFORME DE LA BOX ==================================
#   Tipo  Solicitud                    Dific Estado          T.total    T.real      CPU Meeseeks  Niv  Resolvio (pid,ppid
    ,N,i)
1   texto Quiero que Jerry sea interes   0.00 caos           20.3357s   0.3853s  0.5063s      259    7  -
-------------------------------------------------------------------------------------
Solicitudes: 1 | completadas: 0 | no completadas: 0 | caos: 1   (* dificultad a la suerte)
T.real = tiempo total menos la espera simulada del camino critico.
=====================================================================================
[Box    pid 22196 | ppid 22065] Gracias por usar la Box de Mr. Meeseeks. Hasta luego!
```

10 0x02: experimentos y análisis

10.1 Metodología

Todas las mediciones se tomaron con el programa final, en una máquina virtual Linux (kernel 6.18, 2 núcleos, 8 GB de RAM, `pid_max` = 32 768) y sin otra carga. Cada experimento alimenta la Box por la entrada estándar en modo silencioso (`-q`), para que el costo de imprimir en pantalla no contamine los tiempos, y guarda una fila CSV por solicitud (`-o`). El script `experiments/run_experiments.sh` reproduce todos los experimentos y `experiments/analyze.py` genera las tablas y figuras de esta sección directamente desde los CSV, sin transcribir números a mano. Los tiempos son los definidos en la sección 6.2: T_{total}, T_{sim}, $T_{\text{real}} = T_{\text{total}} - T_{\text{sim}}$ y CPU de los descendientes.

Para que las comparaciones sean válidas, en cada experimento se cambia una sola variable y el resto queda fijo. En los experimentos sobre la dificultad se usa una duración fija y corta (0.1 s) en lugar de la aleatoria: así las diferencias se deben a la dificultad y no al azar de las esperas, y se pueden repetir 20 veces cada caso en tiempos razonables. La duración aleatoria se estudia aparte en E2.

Exp.	Pregunta	Variable
E1	¿Cómo crece el costo con la dificultad? ¿Cuánto cuesta cada Meeseeks?	d (22 valores, 20 rep.)
E2	¿Qué efecto tiene que la duración sea aleatoria y no fija?	duración (10 rep.)
E3	¿Cuánto cuesta en tiempo real cada caso (A, B y C)?	tipo de solicitud
E4	¿Qué efecto tienen las capas del plan de contingencia?	`-a`, `-m`
E5	¿Qué tan sensible es el modelo a la ganancia de dilución?	g
E6	¿Cuánto tiempo extra necesita una tarea que supera el máximo?	d muy pequeña, `-x`

10.2 E1: costo en función de la dificultad

Se enviaron 20 solicitudes de texto por cada uno de 22 valores de dificultad, con duración fija de 0.1 s y la configuración por defecto (128 *slots*, tope de 3000).

d	Éxito	Meeseeks	RIC	Máx.	Niv.	T_{total}	T_{sim}	T_{real}	CPU
100	100 %	1	1–1	1	1	0.101	0.10	0.0008	0.0005
95	100 %	1	1–1	1	1	0.101	0.10	0.0009	0.0005
90	100 %	1	1–1	1	1	0.101	0.10	0.0008	0.0004
86	100 %	1	1–1	1	1	0.101	0.10	0.0008	0.0004
85	100 %	2	1–3	3	2	0.156	0.15	0.0014	0.0010
80	100 %	1	1–3	5	1	0.101	0.10	0.0010	0.0005
70	100 %	3	1–5	5	2	0.212	0.21	0.0019	0.0017
60	100 %	5	3–5	9	3	0.212	0.21	0.0022	0.0024
50	100 %	5	5–8	11	3	0.215	0.21	0.0029	0.0026
46	100 %	8	5–11	13	4	0.333	0.33	0.0033	0.0030
45	100 %	10	8–14	18	3	0.213	0.21	0.0029	0.0039
40	100 %	12	8–16	26	3	0.213	0.21	0.0029	0.0043
30	100 %	14	12–22	31	3	0.214	0.21	0.0041	0.0056
20	100 %	68	54–71	75	4	0.342	0.33	0.0123	0.0243
10	100 %	140	127–174	283	4	0.355	0.33	0.0245	0.0417
5	100 %	288	280–304	321	5	0.507	0.46	0.0470	0.1125
2	100 %	316	292–458	540	5	0.513	0.46	0.0534	0.1246
1	100 %	494	460–639	686	6	0.691	0.60	0.0913	0.2526
0.5	100 %	652	630–768	825	7	0.873	0.75	0.1234	0.3797
0.1	100 %	1070	959–1097	1125	10	1.418	1.17	0.1828	0.7972
0.01	100 %	1581	1504–1627	1777	14	2.108	1.86	0.2764	1.4727
0	0 %	3000	3000–3000	3000	20	13.447	12.05	1.4006	14.1741

Cuadro 1: E1: medianas por dificultad (20 repeticiones). RIC es el rango intercuartílico de los Meeseeks creados. Tiempos en segundos.

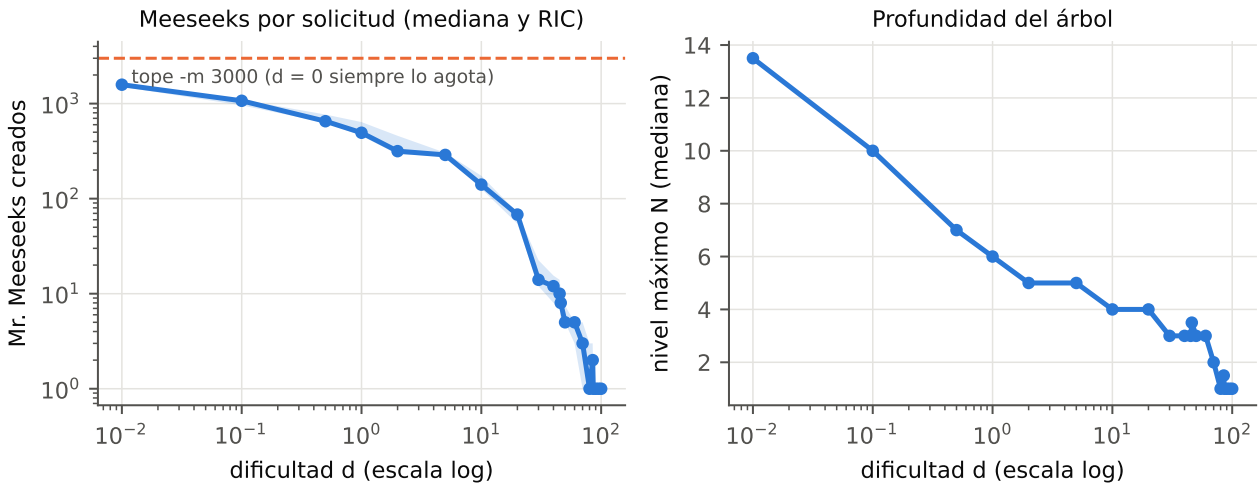


Figura 4: E1: Meeseeks creados y profundidad del árbol frente a la dificultad (ejes logarítmicos en d; las solicitudes con $d = 0$ no aparecen porque $\log 0$ no existe y siempre llegan al tope).

Observaciones.

- Para $d > 85$ siempre basta un Meeseeks y T_{total} es prácticamente la duración simulada: el sistema operativo agrega menos de 1 ms.

- Entre 45 y 85 se crean pocos Meeseeks (entre 1 y 13) gracias al Tiro Meeseeks y a que con 2 ayudantes la dificultad sube rápido. Se nota la discontinuidad del enunciado en $d = 45$: con $d = 46$ se piden 2 ayudantes y con $d = 45$ se piden 5. Con 5 ayudantes se crean algo más de procesos (10 contra 8 en mediana), pero el árbol es menos profundo (3 contra 4 niveles) y la tarea termina antes (0.213 s contra 0.333 s). Más paralelismo cambia procesos por tiempo.
- Por debajo de $d \approx 20$ el costo crece rápidamente: 68 Meeseeks con $d = 20$, 288 con $d = 5$ y más de 1500 con $d = 0.01$. Sin embargo, como la dilución multiplica la facilidad por casi 2.1 en cada generación, la profundidad solo crece como $\log(1/d)$: de 4 niveles con $d = 10$ a 14 con $d = 0.01$. El tiempo total sigue esa misma ley logarítmica.
- Todas las tareas con $d > 0$ se resolvieron (100 %). Todas las de $d = 0$ terminaron en el tope de 3000 Meeseeks, con una mediana de 20 niveles, sin colapsar el sistema.

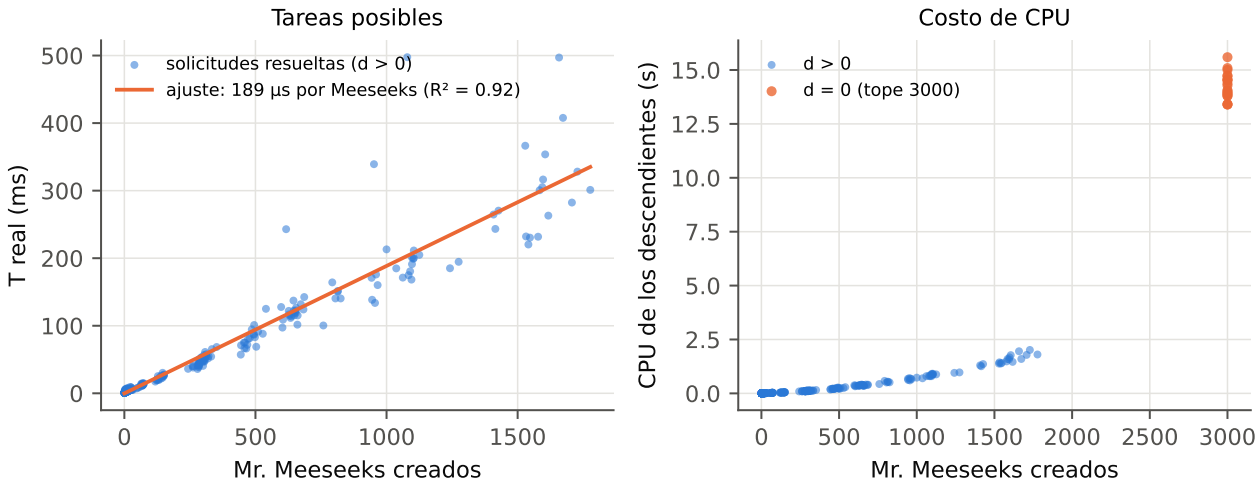


Figura 5: E1: tiempo real (sin esperas simuladas) y CPU frente a la cantidad de Meeseeks creados.

Costo por Meeseeks. En las solicitudes resueltas, el tiempo real crece de forma lineal con la cantidad de Meeseeks: el ajuste da unos 189 µs de tiempo de pared por Meeseeks ($R^2 = 0.92$) y unos 861 µs de CPU. La CPU supera al tiempo de pared porque hay dos núcleos trabajando en paralelo y porque cada `fork()` copia las tablas de páginas del padre. En cambio, las tareas imposibles se salen de la recta (puntos naranja): 3000 Meeseeks consumen alrededor de 14 s de CPU, unas cinco veces lo que predice el ajuste. La causa se analiza en E4.

10.3 E2: duración aleatoria frente a duración fija

Para tres dificultades se comparó la duración aleatoria uniforme en $[0.5, 5]$ s contra una duración fija igual a su media (2.75 s), con 10 repeticiones cada una.

d	Duración	T_{total} (s)	T_{sim} (s)	T_{real} (ms)	Meeseeks	Niv.
90	aleatoria 0.5-5 s	2.90 ± 1.33	2.89	3.5 ± 1.4	1	1
90	fija 2.75 s	2.75 ± 0.00	2.75	4.0 ± 1.5	1	1
50	aleatoria 0.5-5 s	6.44 ± 2.00	6.43	8.2 ± 2.6	5	3
50	fija 2.75 s	7.43 ± 1.74	7.42	9.7 ± 2.1	6	3
20	aleatoria 0.5-5 s	5.53 ± 0.95	5.52	10.9 ± 3.5	28	4
20	fija 2.75 s	8.43 ± 1.40	8.42	18.2 ± 5.6	63	4

Cuadro 2: E2: media $\pm$ desviación estándar; Meeseeks y niveles en mediana.

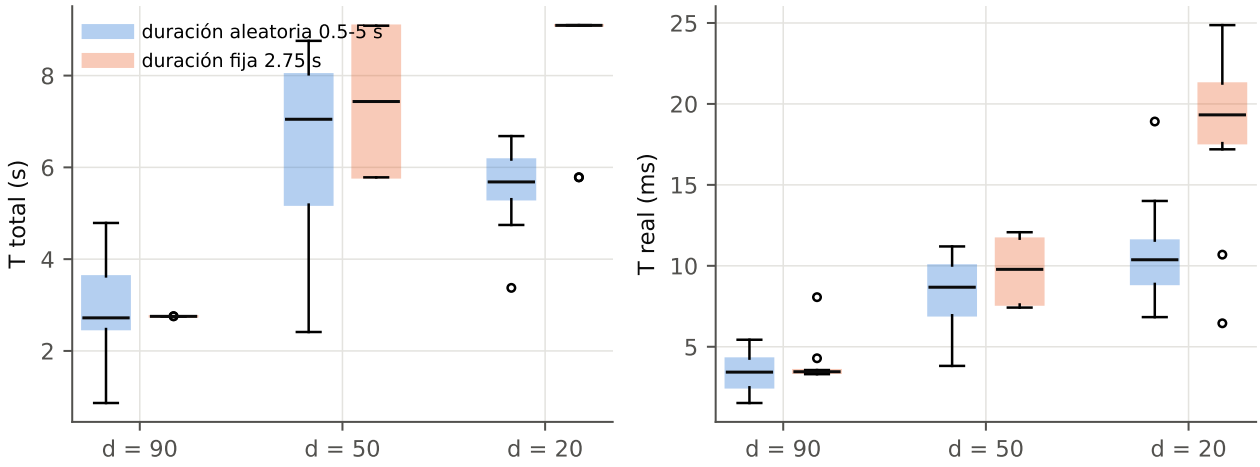


Figura 6: E2: tiempo total y tiempo real según el tipo de duración.

Observaciones.

- El tiempo total está dominado por completo por la espera simulada: T_{total} y T_{sim} difieren en milisegundos. Esto confirma que, para comparar el costo del sistema, hay que mirar T_{real} y no T_{total}.
- Con un solo Meeseeks ($d = 90$) la media es la misma con ambas duraciones (2.90 contra 2.75 s), como se espera, pero la aleatoria tiene una dispersión grande.
- Con muchos Meeseeks ($d = 20$) ocurre algo menos obvio: la duración aleatoria es *más rápida* (5.53 contra 8.43 s) y crea menos de la mitad de los procesos (28 contra 63). Cuando varias ramas trabajan en paralelo, la solución llega por la rama que terminó primero, y el mínimo de varias duraciones aleatorias es mucho menor que su media. Además, al resolverse antes, no alcanzan a nacer las generaciones siguientes. En la serie, un Meeseeks con suerte puede ahorrarle trabajo a todos los demás.
- El tiempo real de un solo Meeseeks es de unos 3.5 ms tras dormir 2.75 s, mientras que en E1, tras dormir 0.1 s, era de 0.8 ms. La diferencia no viene del programa sino de la latencia para despertar: después de varios segundos de inactividad la máquina virtual pone a dormir los núcleos y reactivarlos cuesta algunos milisegundos.

10.4 E3: comparación de los tres casos

Se midió el tiempo real de cada tipo de solicitud que atiende un solo Meeseeks. Para el caso A se usó $d = 100$ con una duración de 1 µs, de modo que el tiempo medido sea prácticamente solo el del mecanismo.

Caso	n	Éxito	Media (ms)	Mediana (ms)	P95 (ms)	CPU (ms)
A: texto d=100 (1 Meeseeks)	200	100 %	0.319	0.276	0.531	0.230
B: calc (válido)	150	100 %	0.282	0.256	0.376	0.278
B: calc (error)	50	0 %	0.253	0.230	0.407	0.254
C: run true	200	100 %	2.400	2.332	2.851	2.253
C: run false	50	0 %	2.133	2.019	2.721	2.135
C: run ls /usr/bin	50	100 %	5.878	5.472	7.382	5.616
C: run sleep 0.25	20	100 %	255.429	255.288	256.103	4.929

Cuadro 3: E3: tiempo real por tipo de solicitud.

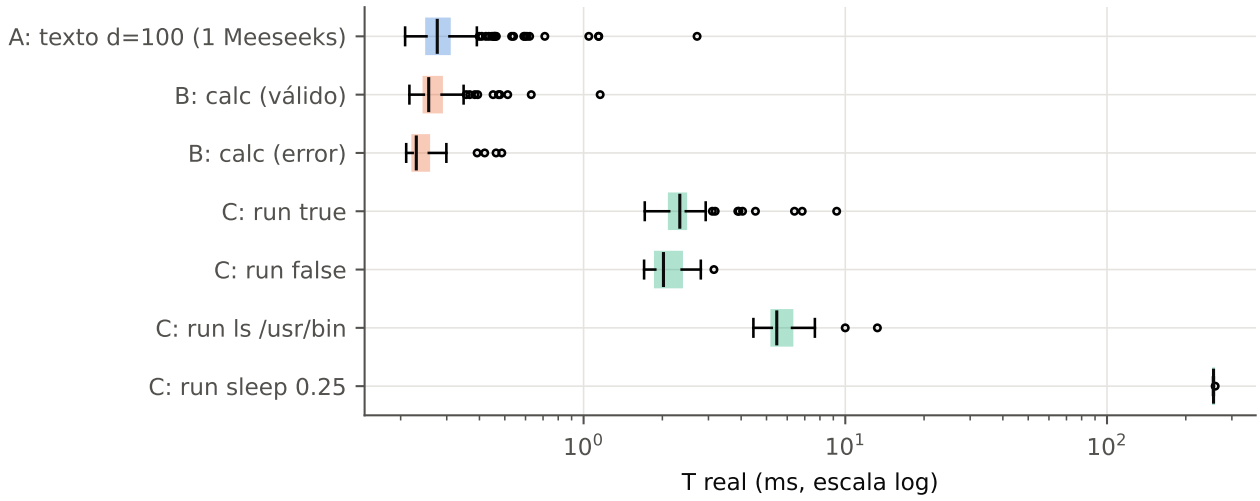


Figura 7: E3: distribución del tiempo real por caso (escala logarítmica).

Observaciones.

- Los casos A (un Meeseeks) y B cuestan lo mismo, alrededor de 0.25 a 0.3 ms: ese es el costo base de crear un proceso con `fork()`, mandarle un mensaje por pipe, recibir la respuesta y recogerlo con `waitpid()`. Evaluar la expresión es despreciable frente a eso, y que la expresión tenga un error no cambia el tiempo.
- El caso C es unas 9 veces más caro (unos 2.3 ms con `true`) porque el Meeseeks hace un segundo `fork()` y un `exec()` de `/bin/sh`, que a su vez carga y ejecuta el programa: cargar un ejecutable nuevo (abrir el archivo, mapear sus segmentos, enlazar bibliotecas dinámicas) cuesta mucho más que duplicar un proceso que ya está en memoria.
- El resultado del programa externo no cambia el costo (`true` y `false` tardan lo mismo); lo que cambia es el trabajo del programa (`ls /usr/bin` tarda más del doble) y, sobre todo, si el programa

espera: con `sleep 0.25` el tiempo real es de 255 ms, pero la CPU es de solo 5 ms. En el caso C no hay espera simulada, así que el tiempo del programa externo es tiempo real de la solicitud.

10.5 E4: plan de contingencia

E4a: semáforo de *slots*. Con $d = 10$ y duración 0.1 s (15 repeticiones) se varió la cantidad de Meeseeks que pueden trabajar a la vez.

Slots (`-a`)	T_{total} (s)	T_{real} (ms)	Meeseeks	Pico vivos	Niv.	CPU (ms)
2	0.605	5.0	9	9	5	10.0
4	0.465	4.8	16	14	5	11.4
8	0.467	6.7	28	24	5	16.1
16	0.468	9.3	44	38	5	23.6
32	0.343	12.9	66	59	4	24.9
64	0.350	20.0	100	92	4	37.6
128	0.353	22.7	142	133	4	44.3
512	0.352	22.1	140	135	4	42.3

Cuadro 4: E4a: medianas según la cantidad de *slots*.

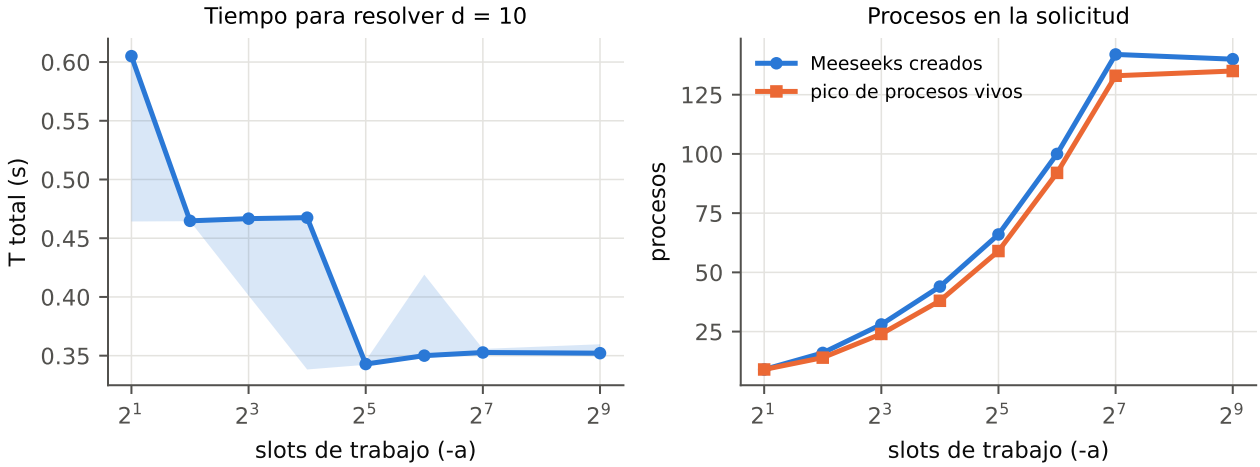


Figura 8: E4a: efecto del semáforo de *slots* con $d = 10$.

Restringir los *slots* reduce mucho los procesos creados (de 142 a 9) y la CPU (de 44 a 10 ms) a cambio de un poco más de tiempo (de 0.35 a 0.61 s). Con pocos *slots* el árbol crece en profundidad en lugar de en anchura: los primeros Meeseeks en conseguir *slot* siguen diluyendo la tarea hacia abajo, y cuando uno la resuelve todavía no han nacido muchos de sus primos. Con muchos *slots* se crean procesos que al final no sirven. Por encima de 128 no hay diferencia porque el árbol nunca necesita más.

E4b: tope total con tareas imposibles. Con $d = 0$ y duración 0.1 s (5 repeticiones) se varió el tope `-m`.

Tope (`-m`)	Éxito	T_{total} (s)	T_{real} (s)	Meeseeks	Pico vivos	Niv.	CPU (s)
100	0 %	0.71	0.013	100	98	4	0.04
250	0 %	1.02	0.021	250	247	5	0.09
500	0 %	1.79	0.038	500	499	7	0.35
1000	0 %	3.89	0.040	1000	999	11	1.56
2000	0 %	8.57	0.071	2000	1999	17	6.01
3000	0 %	13.27	1.818	3000	2994	20	14.07
5000	0 %	29.52	14.569	5000	4951	23	44.30

Cuadro 5: E4b: medianas según el tope de Meeseeks para una tarea imposible.

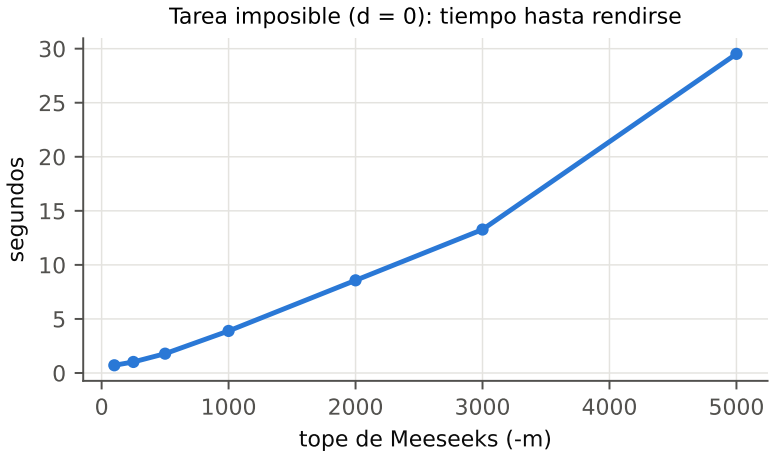


Figura 9: E4b: tiempo hasta rendirse frente al tope.

El tope cumple su función: ninguna tarea imposible pasó del límite y todas terminaron en segundos. Sin embargo, el costo no es lineal en el tope. Al duplicar de 1000 a 2000 Meeseeks la CPU se multiplica por casi 4, y con 5000 el programa usó unos 44 s de CPU en 30 s de pared, es decir, los dos núcleos casi saturados. El pico de procesos vivos es prácticamente igual al tope, porque en una tarea imposible casi nadie termina hasta que se rinde todo el árbol.

La causa es una decisión de diseño. Un padre que quiere crear un ayudante y no encuentra *slot* no se bloquea en `sem_wait`, porque necesita seguir atendiendo su pipe (podría llegarle `M_STOP`); en su lugar vuelve a intentarlo cada 25 ms con `poll()`. Con miles de padres en esa situación, el sistema hace miles de intentos por segundo y la cantidad de intentos crece con la cantidad de padres multiplicada por el tiempo que esperan, lo que da un costo cuadrático. Es una forma de espera activa. Una mejora sería que los Meeseeks que liberan un *slot* despierten a los que esperan por un descriptor (por ejemplo un pipe de fichas o un `eventfd`) que se pueda vigilar con el mismo `poll()`; así la espera sería pasiva sin dejar de atender el pipe.

10.6 E5: sensibilidad a la ganancia de dilución

La ganancia g controla qué tan rápido se simplifica la tarea entre generaciones. Se probaron tres valores con tres dificultades (20 repeticiones, duración 0.1 s).

d	g	Éxito	Meeseeks (med.)	Máx.	Niv.	T_{total} (s)	T_{real} (ms)
50	0.25	100 %	11	29	4	0.333	3.1
50	0.5	100 %	5	13	3	0.212	2.3
50	1	100 %	5	7	3	0.212	2.0
20	0.25	100 %	136	268	4	0.351	21.0
20	0.5	100 %	65	77	4	0.340	10.4
20	1	100 %	14	24	3	0.213	3.1
5	0.25	100 %	488	635	6	0.676	76.1
5	0.5	100 %	285	371	5	0.504	44.0
5	1	100 %	114	135	4	0.348	17.3

Cuadro 6: E5: efecto de la ganancia g (medianas).

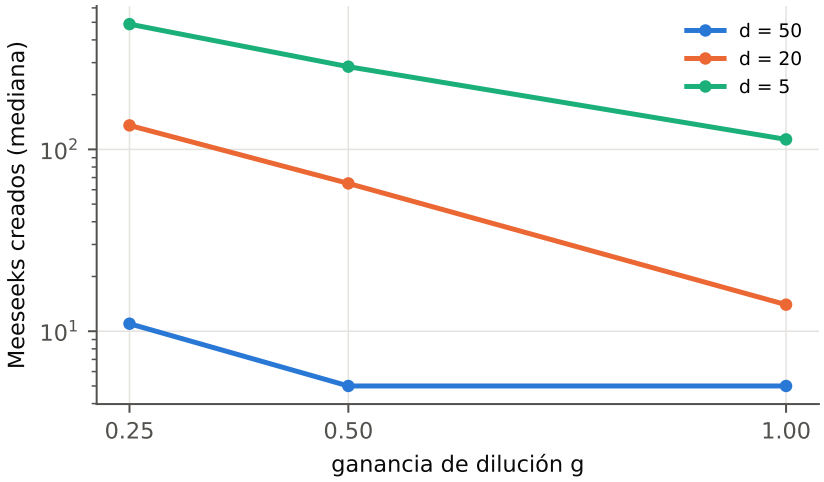


Figura 10: E5: Meeseeks creados según la ganancia.

Con $g = 1$ la dilución es tan rápida que incluso $d = 5$ se resuelve con alrededor de 110 Meeseeks y 4 niveles; con $g = 0.25$ se necesitan unos 490 y 6 niveles. El efecto es mayor para las dificultades medianas ($d = 20$: de 136 a 14 Meeseeks, casi diez veces menos) que para las fáciles. El valor por defecto $g = 0.5$ es un término medio: deja ver árboles grandes para tareas difíciles, como en la serie, sin que las tareas medianas generen cientos de procesos.

10.7 E6: tareas que superan el tiempo máximo

El enunciado pregunta qué pasa si una solicitud muy difícil no se cumple en el tiempo máximo y cuánto tiempo adicional se necesita. Para responderlo se usó el modo experimento (`-x`): al vencer el plazo la Box anuncia el caos pero no termina los procesos, y se mide cuándo llega la solución. Para poder repetir cada caso se trabajó a escala: tiempo máximo de 30 s, 16 *slots*, duración fija de 0.25 s y tope de 30 000 Meeseeks. Se hicieron 3 corridas por dificultad, desde $d = 1$ hasta $d = 10^{-9}$.

d	Resuelta	T_{total} (s)	Extra sobre 30 s	Meeseeks	Niv.	Pico vivos	CPU (s)
1	sí	1.9	0.0	92	7	80	0.07
1	sí	2.3	0.0	101	8	89	0.11
1	sí	2.3	0.0	101	8	92	0.10
0.01	sí	4.7	0.0	184	13	180	0.40
0.01	sí	5.8	0.0	233	15	203	0.53
0.01	sí	4.7	0.0	186	13	181	0.38
0.0001	sí	9.1	0.0	335	20	299	1.21
0.0001	sí	8.4	0.0	322	19	284	1.16
0.0001	sí	9.6	0.0	337	20	313	1.44
1e-06	sí	13.9	0.0	443	25	428	2.72
1e-06	sí	13.0	0.0	436	25	391	2.43
1e-06	sí	14.7	0.0	476	26	448	3.02
1e-09	sí	24.0	0.0	687	35	646	6.79
1e-09	sí	24.9	0.0	724	34	666	6.86
1e-09	sí	351.0	321.0	5059	34	4543	579.12

Cuadro 7: E6: cada corrida con `-T 30 -x -a 16`. "Extra" es el tiempo que pasó después de decretado el caos hasta que llegó la solución.

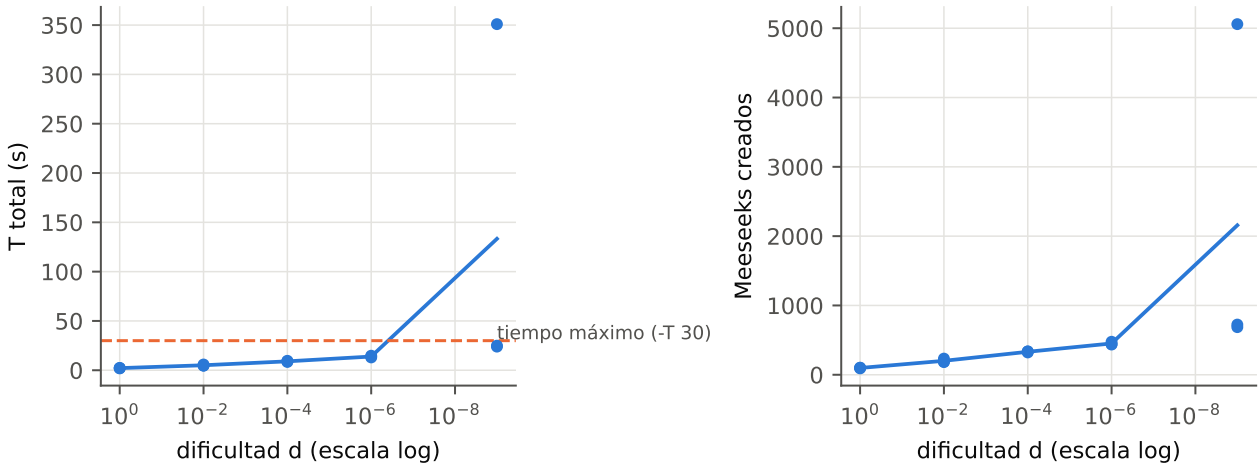


Figura 11: E6: tiempo total y Meeseeks creados para dificultades muy pequeñas.

Observaciones.

- Mientras el sistema no se sature, el costo crece de forma logarítmica: cada factor de 100 en la dificultad agrega unos 5 niveles y unos 4 a 5 s. Incluso $d = 10^{-9}$ se resolvió en dos de tres corridas en unos 24 s, por debajo del máximo, con unos 700 Meeseeks.
- La tercera corrida con $d = 10^{-9}$ es el caso que pide el enunciado: superó los 30 s, se decretó el caos y, sin terminar el programa, la solución llegó a los 351 s, es decir, **321 s adicionales**. Lo notable es que la espera simulada del camino ganador fue de solo 22.5 s: casi todo el tiempo (328 s) fue tiempo real del sistema. Se crearon 5059 Meeseeks, con 4543 vivos a la vez, que consumieron 579 s de CPU en dos núcleos.
- La explicación es la misma que en E4b. Por azar, ese árbol creció más ancho que los otros; con

miles de padres esperando un *slot* y reintentando cada 25 ms, la CPU se satura en espera activa, los Meeseeks que sí trabajan se planifican tarde y el árbol avanza cada vez más lento, lo que a su vez deja más padres esperando. Es una retroalimentación positiva: pasado cierto tamaño, la tarea no se vuelve un poco más lenta, sino un orden de magnitud más lenta.

- Conclusión de la prueba: el tiempo adicional que necesita una tarea muy difícil no es predecible a partir de su dificultad, porque depende de si el árbol cruza el umbral de saturación. Por debajo del umbral bastan segundos; por encima, varios minutos. Esto justifica que el tiempo máximo exista y que su acción sea terminar todo el grupo de procesos: seguir esperando puede costar mucho más que reintentar la solicitud.

A escala real (300 s de máximo y duraciones de 0.5 a 5 s) cada generación tarda en promedio 2.75 s en lugar de 0.25 s, por lo que el mismo árbol necesitaría unas 11 veces más tiempo simulado: una tarea con $d = 10^{-9}$ (unos 34 niveles) pasaría de los 300 s solo por las esperas, aun sin saturación. Por esa razón la prueba se hizo a escala y no con los 5 minutos completos, que habrían requerido horas por corrida. La terminación por caos con un tiempo máximo real se muestra en la corrida de la sección 9.

11 Conclusiones

- Se implementó en un único programa en C la Box de Mr. Meeseeks con los tres tipos de solicitud. Todos los procesos ejecutan el mismo código, se crean con `fork()`, se comunican solo por pipes y comparten contadores protegidos por semáforos POSIX en memoria compartida.
- La Ley de Dilución logística, con $d = 0$ como punto fijo, cumple a la vez dos requisitos que parecen opuestos: cualquier tarea posible termina resolviéndose y una imposible jamás. Los experimentos muestran que la profundidad crece como $\log(1/d)$ y que la cantidad de Meeseeks crece muy rápido al acercarse a 0, como en la serie.
- La concurrencia trajo problemas reales, no solo teóricos. Varios Meeseeks podían resolver la tarea a la vez (se corrigió con una victoria exclusiva en sección crítica) y el semáforo de *slots* produjo un interbloqueo que cumplía las cuatro condiciones de Coffman (se corrigió liberando el *slot* al pasar a esperar). Ninguno de los dos se habría notado sin probar con tareas difíciles.
- Crear un Meeseeks cuesta unos 0.25 ms y algo menos de 0.2 ms adicionales por cada uno en un árbol grande; ejecutar un programa externo cuesta unas 9 veces más por el `exec`. En todas las solicitudes, el tiempo total lo domina la espera simulada, por eso el tiempo real debe medirse restando la espera del camino crítico y no la suma de todas las esperas.
- Cuando hay paralelismo, la aleatoriedad ayuda: con duración aleatoria la tarea se resolvió antes y con menos procesos, porque gana la rama más rápida.
- El plan de contingencia funcionó en todas las pruebas: ninguna tarea imposible colapsó el sistema. Aun así, la espera por *slots* con sondeo periódico crece en forma cuadrática y llega a saturar la CPU con miles de procesos; una espera pasiva sería la siguiente mejora.
- El tiempo máximo no es solo una protección contra tareas imposibles. Las tareas muy difíciles pero posibles también pueden superarlo, y entonces terminar el programa significa perder una solución que estaba en camino (sección 10.7). Por eso el tiempo máximo es una decisión sobre cuánto se está dispuesto a esperar, y su valor debe considerar la profundidad que puede alcanzar el árbol.

Referencias

[1] A. Silberschatz, P. B. Galvin y G. Gagne, *Operating System Concepts*, 10.ª ed. Hoboken, NJ, EE. UU.: Wiley, 2018. Caps. 3 (procesos), 6 y 7 (sincronización) y 8 (interbloqueos).

[2] N. Matthew y R. Stones, *Beginning Linux Programming*, 4.ª ed. Indianápolis, IN, EE. UU.: Wiley, 2008. Caps. 11 (procesos y señales), 13 (comunicación entre procesos: pipes) y 14 (semáforos, memoria compartida y colas de mensajes).

[3] M. Kerrisk, *The Linux Programming Interface*. San Francisco, CA, EE. UU.: No Starch Press, 2010. Caps. 24 a 26 (`fork`, `exit`, `wait`), 44 (pipes), 53 (semáforos POSIX) y 63 (`poll`).

[4] E. G. Coffman, M. Elphick y A. Shoshani, "System deadlocks", *ACM Computing Surveys*, vol. 3, n.º 2, pp. 67–78, 1971.

[5] The Linux man-pages project, `fork(2)`, `pipe(7)`, `sem_overview(7)`, `poll(2)`, `prctl(2)`, `getrusage(2)`. [En línea]. Disponible: `https://man7.org/linux/man-pages/`

[6] J. Roiland y D. Harmon (creadores), "Meeseeks and Destroy", *Rick and Morty*, temp. 1, ep. 5. Adult Swim, 2014.